

Abstract

The detrimental effect of cyber attacks on health, economy, and environment reached a new level with the development and expansion of industrial control systems. Anomaly detection became one of the vital components in such systems to prevent harm. Although signature-based anomaly detectors have managed to mitigate the risk to some extent, they remain susceptible to zero-day attacks. To overcome this, various machine learning-based anomaly detectors have been introduced. However, the lack of labeled training data creates challenges for predictive machine learning models. In this thesis, two state-of-the-art GAN-based anomaly detectors, TadGAN and MADGAN, are used as a starting point to implement a new multivariate reconstruction-based anomaly detector. In addition, the window size and various data preprocessing steps are analyzed and tested with the aforesaid anomaly detector. The tests are conducted with the SWaT dataset which is collected from a complex six-stage secure water treatment system. The results are examined and compared with two other multivariate reconstruction-based anomaly detectors, MADGAN and GAN-AD, as the benchmark.

Contents

Acknowledgments	v
Abstract	vii
1 Introduction	1
1.1 Motivation	1
1.2 Thesis Structure	5
2 Anomaly Detection in Industrial Control Systems	7
2.1 Characteristics of Industrial Control Systems (ICS)	7
2.2 Anomaly Detection in Industrial Control Systems	8
2.3 Data Preprocessing	8
2.3.1 Feature Selection	9
2.3.2 Feature Extraction	12
2.3.3 Long Short Term Memory (LSTM)	14
2.3.4 Generative Adversarial Network	15
2.3.5 Anomaly Detection using both Discrimination and Reconstruction	17
2.4 Dataset	21
2.4.1 SWaT	21
2.4.2 Cyber-Attacks	21
3 Literature Review	23
3.1 TadGAN: Time Series Anomaly Detection Using Generative Adversarial Networks	23
3.2 MAD-GAN: Multivariate Anomaly Detection for Time Series Data with Generative Adversarial Networks	24
3.3 A GAN-based Anomaly Detection Approach for Imbalanced Industrial Time Series	26
3.4 Anomaly Detection with Generative Adversarial Networks for Multivari- ate Time Series	26
3.5 Adversarially Learned Anomaly Detection	27
3.6 High-Performance Unsupervised Anomaly Detection for Cyber-Physical System Networks	28
3.7 Anomaly Detection in Cyber Physical Systems using Recurrent Neural Networks	29
3.8 State-Aware Anomaly Detection for Industrial Control Systems	29
3.9 Fence GAN: Towards Better Anomaly Detection	30

3.10	Anomaly Detection for a Water Treatment System Using Unsupervised Machine Learning	30
3.11	DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning	31
3.12	Detecting Cyberattacks in Industrial Control Systems Using Convolutional Neural Networks	32
3.13	Multi-level Anomaly Detection in Industrial Control Systems via Package Signatures and LSTM networks	33
3.14	Time Series Analysis: Unsupervised Anomaly Detection Beyond Outlier Detection	33
3.15	EDMAND: Edge-Based Multi-Level Anomaly Detection for SCADA Networks	34
3.16	Neural Contextual Anomaly Detection for Time Series	35
4	Methodology	37
4.1	Data Preprocessing	37
4.1.1	SWaT Dataset	37
4.2	Feature Selection and Extraction	40
4.2.1	Feature Selection	40
4.2.2	Feature Extraction	43
4.3	Data Preparation	48
4.4	Dynamic Construction of the Neural Network Architecture	50
4.5	GAN Training and Reconstruction of the Test Data	52
4.5.1	GAN Training	55
4.5.2	Reconstruction of the Test Data	55
4.6	Anomaly Score Computation	56
5	Evaluation and Results	59
5.1	MADGAN and GAN-AD for Benchmark	63
6	Discussion	65
6.1	Limitations	66
6.2	Future Works	66
7	Conclusion	69
	List of Figures	71
	Bibliography	73

1 Introduction

1.1 Motivation

Continuous digitalization and automation are the driving forces behind today's industrial developments. The coupling of sensors and actuators with cutting-edge data analytics and control algorithms enables the emergence of cyber-physical systems (CPS). These are distinguished by the existence of a closed feedback loop between the physical environment and their digital control systems. For a long period of time, similar systems have existed for plant control and internal industrial automation. Additionally, known as Supervisory Control and Data Acquisition (SCADA), these systems were intended to function in isolated locations, ensuring the highest level of security possible. These ecosystems are no longer isolated. Rather than that, even the smallest sensors and actuators are linked to other internal networks, the internet, and external cloud services [1].

Moreover, CPSs are massive, sophisticated, and equipped with networked sensors and actuators that create huge volumes of multivariate time series data that may be utilized to continually monitor the CPS's operating parameters in order to discover and address time-related problems. With the advent of the Internet of Things (IoT), the widespread use of networked sensors and actuators in CPSs and other systems will only become more prevalent, resulting in multiple systems and devices communicating and possibly performing a wide variety of tasks autonomously over networks. While the number of these linked cyber-physical systems continues to grow, they confront growing information security risks. Due to the fact that many CPSs are designed to perform mission-critical functions, they are great targets for cyber-attacks. Thus, it is critical to regularly monitor these systems' activities for intrusion events by utilizing anomaly detection on the multivariate time-series data provided by the systems [2]. Significant hazards may arise as a result of cyberattacks aimed at influencing the system's core physical processes, such as in the Stuxnet attack. Fundamentally, approaches for detecting anomalies must describe the distribution of normal data, which is often complicated and multidimensional. While anomaly detection is a well-studied subject, establishing effective algorithms for complex and multidimensional data continues to be a difficulty [3].

One way of detecting these new and undiscovered dangers is to look for network anomalies [1]. Anomaly detection is a critical challenge in a variety of real-world contexts, including cyber-security, manufacturing, fraud detection, and medical imaging [3]. It is critical for contemporary industrial systems to function reliably and safely. Appropriate and timely identification of anomalies may aid in the prevention of tragic

accidents and boost production efficiency [4].

Typically, an anomaly is characterized as a period in time when the system's behavior deviates considerably from its prior normal state. Thus, the fundamental aim of anomaly detection is to ascertain the time intervals during which an abnormality occurred [2]. In general, industrial anomaly detection is based on physical signals collected over time by diverse sensors, such as current and temperature, which are often referred to as time series [4]. A time-series anomaly is a moment or a time interval when a system exhibits abnormal behavior. Anomalies are broadly classified into two types: A point anomaly is a single data point that has achieved an exceptional value. On the other hand, a collective anomaly is a continuous succession of data points that are regarded as abnormal collectively, even if the individual data points are not [5].

Time series data is often used to train models for industrial anomaly detection. When time series are used as input, a popular framework for anomaly detection often consists of two stages: feature extraction and anomaly identification. Time series are preprocessed into low-dimensional feature vectors by feature extraction algorithms, which are then supplied into an anomaly detector for detection [4]. Time series anomaly detection tries to identify time series with anomalous subsequences of varying durations. Thresholding is a basic detection method that recognizes data items that exceed a usual range. Many anomalies, on the other hand, may not cross any limits — they may have values that are seemingly "normal," but are unusual at the timeframe — they occur such contextual anomalies. These abnormalities are more difficult to detect since the signal's context is often unknown.

Numerous statistical techniques have been suggested to improve thresholding, including Statistical Process Control (SPC), in which data points are flagged as anomalies if they fail statistical hypothesis testing. However, a substantial amount of human expertise is necessary to define the models' previous assumptions [5]. Historically, SPC techniques such as cumulative sum (CUSUM), exponentially weighted moving average (EWMA), and Shewhart charts were often used to monitor the quality of industrial processes and to identify out-of-range operating conditions. These established detection algorithms are incapable of dealing with the multivariate data streams created by the current CPSs' growing dynamic and complicated character. As a result, researchers have progressed beyond specification or signature-based strategies and attempted to use machine learning techniques to leverage the massive volumes of data created by industrial control systems [2].

Detecting abnormalities in time series remains difficult, even with the application of machine learning methods. To begin with, the majority of supervised algorithms need sufficient relevant normal data and labeled anomaly classes for learning, which is seldom the case in reality because of the rarity of anomalies. Second, although the majority of current unsupervised algorithms are constructed using linear projection and transformation, the hidden underlying correlations in multivariate time series of complex CPSs are often non-linear [2]. Additionally, the majority of existing algorithms make use of basic comparisons between the current state and anticipated normal ranges,

which may be insufficient for anomaly detection since the control boundaries are not sufficiently flexible to detect indirect intrusions [6].

Additionally, as a strong pattern recognition tool for anomaly detection, machine learning techniques, such as Bayesian classifiers, support vector machines (SVM), neural networks, and deep learning approaches, have gained attention. However, since all of the aforementioned approaches are based on a class-balanced hypothesis, they all struggle to achieve good classification accuracy with unbalanced data. In many industrial systems, samples taken under abnormal operating circumstances are often inadequate in size. Additionally, when a system operates normally for an extended length of time followed by a sudden aberration, pinpointing the commencement of the abnormality is particularly difficult. As a result, incorrect aberrant labeling will have a detrimental effect on the previously unbalanced scenario. When the labels for normal and abnormal are skewed, classifiers using machine learning techniques assure the accuracy of the majority classes by sacrificing the minority classes, which implies that diagnostic findings for all testing samples will be biased toward normalcy. However, when it comes to industrial system anomaly detection, the emphasis should be squarely on those minority groups [4].

Moreover, researchers examined a variety of unsupervised machine learning algorithms for anomaly detection. One widely used technique involves segmenting a time series into subsequences (overlapping or not) of a predetermined length and then using clustering algorithms to identify outliers. Another method involves developing a model that predicts or reconstructs a time-series signal and comparing the actual and expected or reconstructed values. Anomalies are indicated by large predictions or reconstruction failures [5]. Deep learning approaches are exceptionally capable of managing non-linearity in complicated temporal correlations and have an outstanding capacity for learning. As a result, they have been included in a variety of time series anomaly detection techniques, including those developed by corporations commercially [5].

The generative adversarial networks (GANs) are a kind of deep neural network that has been successfully utilized to represent such complicated and high-dimensional distributions [7]. Nonetheless, GAN framework was shown to be effective at generating time series sequences, either for polyphonic music generation using recurrent neural networks as generators and discriminators, or for real-valued medical time series generation using a conditional version of recurrent GAN. These early results of GAN in creating realistic and complicated datasets, as well as the adversarial training of both a generator and a discriminator, are very indicative of the framework's application to anomaly detection [2]. GANs, in their most basic form, are made up of two competing deep neural networks, a generator G and a discriminator D . D models the data by learning a mapping between latent random variables z (taken from Gaussian or uniform distributions) and the data space, while D learns to differentiate between actual data and reconstructed data [3]. Intuitively, a GAN that has been trained to match the distribution of normal samples should be capable of reconstructing such a sample from

a latent representation and also discriminating the sample as being from the original data distribution [7].

Additionally, it has been shown that GANs are very effective in reconstructing time series sequences as well. However, since GANs only model the data distribution implicitly, employing them for anomaly detection requires a time-consuming optimization technique to recover the latent representation of a specific input sample, making this an unrealistic solution for big datasets or real-time applications [7]. GAN-based approaches such as AnoGAN, BiGAN, and GANomaly train models only on normal pictures, discriminating abnormal images based on the distribution difference between normal and abnormal. In this way, these GAN-based models are good at preventing diagnostic outcomes from being biased toward normalcy when used with imbalanced datasets. However, GAN-based approaches for anomaly detection in industrial applications remain uncommon [4].

As a solution to the above-mentioned problems such as detecting new and undiscovered cyber attacks, fast detection of anomalies, establishing algorithms for complex and multidimensional data, class imbalance of training data, or lack of labeled data, this thesis researches an enhancement to the current state-of-the-art methodologies merging MADGAN and TadGAN. By redesigning the LSTM-based TadGAN as a multivariate anomaly detector, it is intended to use both the generator and discriminator losses in calculating the anomaly score. By adapting MADGAN's multivariate generative adversarial neural network architecture to TadGAN, the aim is to utilize the TadGAN pipeline to detect anomalies by leveraging cycle consistency and Wasserstein loss to improve the accuracy of learning complicated multivariate patterns in the training dataset. Along with the novel anomaly detection architecture, the implications of different feature extraction and feature selection strategies are evaluated.

To assess the novel technique, the SWaT dataset is utilized, which is derived from a complicated six-stage Secure Water Treatment (SWaT) system, to differentiate abnormal attack settings from typical functioning conditions. To verify this strategy, MADGAN and GAN-AD are implemented as described in the publications MAD-GAN: Multivariate Anomaly Detection for Time Series Data with Generative Adversarial Networks and Anomaly Detection using Generative Adversarial Networks for Multivariate Time Series, respectively, which are both trained using the same dataset to develop the control group.

1.2 Thesis Structure

The content of the rest of this thesis for each chapter is described as follows:

Chapter 2 explains the theory behind the technology used to implement the anomaly detection algorithm for industrial control systems. The chapter entails the characteristics of industrial control systems, anomaly detection using time series, and how these elements can come into play to implement a GAN-based anomaly detector.

Chapter 3 describes the related works as background knowledge that is valuable for the reader to understand the described concepts better in the following chapters.

Chapter 4 describes how a GAN-based anomaly detector is implemented, trained using multivariate time series data, and tested. This chapter also explains how feature selection and feature extraction techniques are employed during the preprocessing of the data. The chapter provides information about the evaluation metrics alongside the SWaT dataset and cyber attacks.

Chapter 5 presents the results obtained from the tests. The proposed anomaly detector is compared with the MADGAN and GAN-AD for validation.

Chapter 6 discusses the test results presented in the previous chapter and offers future research topics based on the encountered challenges and weaknesses of the algorithm.

Chapter 7 concludes the thesis by summarizing the research.

2 Anomaly Detection in Industrial Control Systems

2.1 Characteristics of Industrial Control Systems (ICS)

ICSs are found in facilities such as those that treat and distribute water, as well as those that generate, transmit, and distribute electricity. The complexity of an ICS enhances an attacker's attack surface, allowing him to target both the cyber and physical components of a facility. Control software in an ICS may also be vulnerable for a variety of reasons, including unpatched or incompletely patched legacy code, the absence of industry-standard security certifications for ICS devices, and a lack of resources to maintain the ICS updated.

ICS are made up of supervisory control and data acquisition systems and distributed control systems. As shown in the figure ref 2.1, the control system is composed of a number of PLCs, each regulating a different aspect of the physical process. Each PLC interacts with a group of sensors and actuators through a local network, which is a multi-layer network, also known as a field-bus network. The PLCs interact over a network.

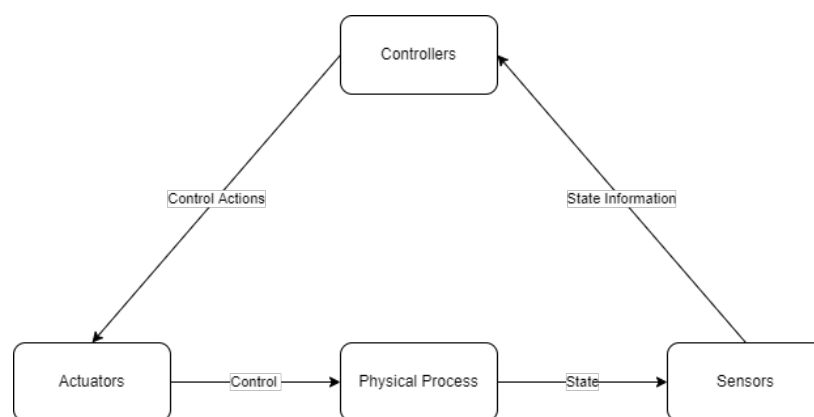


Figure 2.1: ICS Architecture

The terms "operational technology" (OT) refer to supervisory control and data acquisition systems and distributed control systems. Convergence between information technology (IT) and operations technology is growing in industrial control systems. As a result of this convergence, OT data may now be accessed from an IT environment,

for example, through remote access. OT data contains vital plant information such as temperatures, level indications, control signals, sensor signals, and actuator statuses. This is especially true for water distribution systems, which are dispersed throughout a city, making them an ideal target for cyber-physical attacks [8].

2.2 Anomaly Detection in Industrial Control Systems

A signature-based IDS identifies malicious activity in the network traffic or application events of a SCADA system. It does this by comparing suspicious events to a database of known attack signatures or fingerprints. The false positive rate, such as wrongly classifying a benign occurrence as an attack, is very low and may approach zero with this sort of IDS. Additionally, the detection time may be reduced since the detection phase is only reliant on the usage of a matching procedure. Despite the benefits of signature-based IDSs, they often fail to identify novel attacks like zero-day whose signatures are unknown or do not exist in their database. As a result, the database must be regularly updated with new attack patterns.

An anomaly-based IDS presupposes that intrusive activities exhibit characteristics that are markedly different from typical activities. The "normal model" is constructed by utilizing sophisticated mathematical and statistical approaches on a realistic training dataset. Any variation from this model that is substantial is noted as an anomaly or possible intrusion. For example, typical SCADA network traffic may be acquired during a period of regular operations, and then normal SCADA network profiles can be created using a modeling approach. The divergence degree between the current network traffic and the reconstructed network profile is calculated during the detection phase; if the divergence exceeds a predetermined threshold, the current network traffic is reported as intrusive activity. The fundamental benefit of anomaly-based IDSs over signature-based ones is that they may identify novel or undiscovered threats. However, they often have a higher false positive rate, detecting normal behavior as malicious [9].

2.3 Data Preprocessing

It is challenging to train anomaly-based IDSs, specifically GANs. They are challenging to train since both the generator and discriminator models are trained concurrently in a zero-sum game. This implies that enhancements to one model are made at the cost of the other. The objective of training two models is to arrive at a point of balance between two conflicting concerns. Additionally, each time one of the models' parameters is altered, the nature of the optimization issue being addressed changes. This results in the formation of a dynamic system.

In neural network terminology, the technical difficulty of simultaneously training two competing neural networks is that they may fail to converge. Rather than converge, GANs may succumb to one of a few possible failure scenarios. A typical failure scenario is for the generator to fluctuate between creating particular instances in the domain

rather than reaching a point of equilibrium. Perhaps the most perplexing model failure is when many inputs to the generator produce the same output.

Feature selection and feature extraction are fundamental topics in machine learning that have a significant influence on the model's performance. The features employed to train machine learning models have a significant impact on the performance that may be achieved. Features that are irrelevant or just partly relevant might have a detrimental effect on model performance. The first and most critical stage in developing a model is feature selection and feature extraction [10].

2.3.1 Feature Selection

A feature selection method may be thought of as a mix of a search approach for suggesting new feature subsets and an assessment metric for scoring the various feature subsets. The most straightforward approach is to evaluate each potential subset of characteristics and choose the one with the lowest error rate. This is a thorough search of the space, and for all but the smallest feature subsets, it is computationally expensive. The evaluation measure used has a significant impact on the algorithm, and it is these evaluation metrics that differentiate the three primary types of feature selection algorithms: wrappers, filters, and embedding techniques [11].

Pearson Correlation Coefficient

The Pearson correlation coefficient is a statistic that indicates the strength of a linear link between two variables. This wrapper type feature selection technique is symbolized by the symbol r . In essence, a Pearson correlation tries to draw the bestline through the data of two variables, and the Pearson correlation coefficient indicates how far apart all of these data points are from this line. In other words, it explains how well the data points fit into this new model of best fit.

Pearson's correlation coefficient, r , may vary between +1 and -1. A correlation coefficient of 0 shows that there is no relationship between the two variables. A number larger than 0 implies a positive connection; that is, when one variable's value grows, the other variable's value grows as well. A number less than 0 implies a negative correlation; that is, when one variable's value grows, the other variable's value falls. The higher the correlation between the two variables, the closer the Pearson correlation coefficient will be to +1 or -1, depending on the direction of the link. A number of +1 or -1 indicates that all of the data points are included on the line of best fit and no data points deviate from this line. The closer r is to 0, the more variance there is around the line of best fit [12].

Chi-Squared

A chi-square statistic is a comparison statistic that indicates how well a model fits real, observed data. The data required to calculate the chi-square statistic must be random,

unstructured, mutually exclusive, derived from independent variables, and gathered from a sufficiently large sample. For instance, the outcome of a fair coin flip meets these conditions.

Chi-square tests are often used to assess hypotheses. The chi-square statistic is used to assess the magnitude of any disparities between predicted and observed findings, taking into account the sample size and the number of variables in the association.

$$X_c^2 = \sum (O_i - E_i)^2 / E_i \quad (2.1)$$

where:

C denotes the number of degrees of freedom,

O denotes the observed value(s),

E denotes the expected value(s).

Degrees of freedom are used in these tests to establish whether or not a certain null hypothesis can be rejected based on the overall number of variables and samples in the experiment. As is the case with every statistic, a bigger sample size resulted in more accurate findings.

To get the chi-squared value for feature selection, the X^2 difference between each feature and the objective is computed. Following that, the appropriate number of features with the highest X^2 scores are chosen. The underlying assumption is that if a characteristic is unrelated to the aim, it is useless for categorizing data [13].

Recursive Feature Elimination (RFE)

RFE is a feature selection algorithm of the wrapper type. This implies that a distinct machine learning algorithm is provided and employed at the method's core, wrapped with RFE, and used to assist in feature selection. This is in contrast to feature choices based on filters, which score each feature and choose the ones with the highest (or lowest) score. It is a wrapper-style feature selection technique that internally employs filter-based feature selection. RFE works by successfully eliminating features from the training dataset until the target number remains. This is accomplished by fitting the model using the supplied machine learning method, ranking features according to their relevance, deleting the least important features, and re-fitting the model. This procedure is repeated until only a certain number of characteristics remain. Features are either assessed using the machine learning model offered, such as logistic regression, or using a statistical technique [14].

In practice, the analyst defines the number and size of feature subgroups to be analyzed. As a result, the size of the subset is a tuning parameter for RFE. The subset size that maximizes the performance criterion is utilized to rank the features. Following that, the optimum subset is utilized to train the final model. Not all models are compatible with RFE, and some models benefit from RFE more than others. Due to the fact that RFE requires the initial model to employ the whole feature set, some models cannot be used when the number of features exceeds the number of samples [15].

Random Forest

Random forests are composed of between four and twelve hundred decision trees, each of which is constructed using a random extraction of observations from the dataset and a random extraction of features. Because not every tree sees all characteristics or all observations, the trees are de-correlated and hence less susceptible to over-fitting. Additionally, each tree contains a series of yes-no questions depending on a single or a combination of attributes. At each node, the tree partitions the dataset into two buckets, each of which contains observations that are more similar to one another yet distinct from the others. Thus, the significance of each attribute is determined by the "purity" of the buckets. Impurity is quantified in classification using either the Gini impurity or the information gain/entropy. Variance is used to quantify impurity in regression.

Thus, while training a tree, it is feasible to determine the amount by which each feature reduces impurity. The more an attribute contributes to the reduction of impurity, the more significant the attribute is. In random forests, the reduction in impurity caused by each feature may be averaged across trees to estimate the variable's ultimate significance. To provide a better understanding, characteristics picked at the top of the trees are normally more relevant than features selected at the tree's end nodes, since top splits typically result in larger information gains [16].

Lasso

Lasso regression is a kind of linear model that makes use of the following cost function:

$$\frac{1}{2N_{\text{training}}} \sum_{i=1}^{N_{\text{training}}} (y_{\text{real}}^{(i)} - y_{\text{pred}}^{(i)})^2 + \alpha \sum_{j=1}^n |a_j| \quad (2.2)$$

The coefficient of the j -th feature is denoted by a_j . The last term is referred to as the L1 penalty, and is a hyperparameter that controls the strength of this penalty term. The greater the coefficient of a characteristic, the greater the cost function's value. Thus, the purpose of Lasso regression is to minimize the cost function by lowering the coefficients' absolute values. This works only if the features have been scaled earlier, for example, by standardization or another scaling approach. A cross-validation strategy must be used to determine the hyperparameter value.

By attempting to minimize the cost function, Lasso regression will automatically choose the most valuable features, rejecting the redundant or worthless ones. Discarding a feature in Lasso regression results in its coefficient being equal to 0. Thus, the concept of using Lasso regression to pick features is rather straightforward: Lasso regression is run on a scaled version of the dataset and only those features with a coefficient greater than 0 are considered. But first, the hyperparameters are modified to get the desired kind of Lasso regression [17].

2.3.2 Feature Extraction

Feature extraction is the process of extracting higher-order features from the original features by utilizing expert knowledge. The required activities are outlined to get those higher order characteristics in this part, keeping in mind that ICSs are designed to handle industrial and vital operations that routinely execute repeated tasks. For instance, an ICS that controls and monitors a water tank should take into account the water tank's lower and higher limitations while opening and closing the input and output water valves.

This process is repeated when the water reaches its lowest and highest points. These recurring operations are represented in the log files used to generate the datasets. Collecting these features from the repeating activities is possible using autoencoders and PCA [18].

Principal Component Analysis (PCA)

PCA may be thought of as fitting the data to a p -dimensional ellipsoid, with each axis representing a principal component. If the variance along one of the ellipsoid's axes is tiny, the variance along that axis is likewise minimal.

To determine the ellipsoid's axes, the values of each variable needs to be centered in the dataset on 0 by subtracting the variable's mean from each of those values. These modified values are used in place of the variables' original observed values. Then, the data's covariance matrix is constructed with its eigenvalues and accompanying eigenvectors. Next, each of the orthogonal eigenvectors must be normalized to create unit vectors. Once this has been accomplished, each of the mutually orthogonal unit eigenvectors may be regarded as one axis of the fitted ellipsoid. This basis choice converts the covariance matrix to a diagonalized version, with the diagonal components reflecting the variance of each axis. The percentage of variance represented by each eigenvector may be computed by dividing the associated eigenvalue by the total of all eigenvalues.

$T = XW$ transforms a data vector x_i from an initial space of p variables to a new space of p variables that are uncorrelated throughout the dataset. However, not all of the primary components must be retained. The truncated transformation is obtained by retaining just the first L principal components, which are obtained by using only the first L eigenvectors.

$$T_L = XW_L \quad (2.3)$$

where T_L now has n rows but only L features. By design, this scoring matrix maximizes the variation in the original data that has been maintained while minimizing the overall squared reconstruction error.

This kind of dimensionality reduction may be very beneficial for visualizing and analyzing large datasets while maintaining as much of the dataset's volatility as feasible. For example, choosing $L = 2$ and retaining only the first two principal components identifies the two-dimensional plane through the high-dimensional dataset in which the

data is most spread out; if the dataset shows clusters, these may also be most scattered out and thus most observable to be visualized in a two-dimensional diagram; whilst, if two random directions through the data or two of the original variables are chosen, these clusters may be much less scattered apart.

Similarly, the more explanatory variables permitted in regression analysis, the higher the likelihood of overfitting the model, resulting in findings that do not generalize to other datasets. One strategy, particularly when there are high correlations between the many explanatory variables, is to reduce them to a few main components and then run the regression against them, a technique known as "principal component regression".

Reduced dimensionality could also be necessary when the variables in a dataset are noisy. If each column of the dataset includes independent Gaussian noise with an equivalent distribution, then the columns of T will likewise contain Gaussian noise with an equal distribution. However, with a larger concentration of total variance in the first few main components relative to the same noise variance, the proportional influence of noise is reduced, and the first few components obtain a higher signal-to-noise ratio. So, PCA may have the effect of condensing a large portion of the signal into the first few principal components, which can be captured advantageously through dimensionality reduction, whereas the latter principal components may be influenced by noise and thus discarded without significant loss [19].

Autoencoder

An autoencoder is a type of neural network that attempts to learn a compact representation of an input. Typically, autoencoders are trained as part of a larger model that tries to replicate the input. In simple terms, an autoencoder (AE) consists of two components: an encoder and a decoder. With a dataset X having samples and features, the encoder's output Y gives the reduced representation of X , and the decoder is configured to reconstruct the original dataset X from the encoder's representation Y by minimizing the difference between X and X' . The autoencoder model's design purposefully complicates this by limiting the architecture to a bottleneck at the model's midway, from where the input data is reconstructed. To be more precise, the encoder is a function f that converts an input X to a hidden representation Y :

$$Y = g(Y) = s_f(WX + b_X) \quad (2.4)$$

If s_f is a nonlinear activation function, the encoder is parameterized using a weight matrix W and a bias vector b in R^n . The decoder function g converts a concealed representation Y to a reconstruction X' :

$$X' = g(Y) = s_g(W'Y + b_Y) \quad (2.5)$$

s_g is the activation function of the decoder, which is commonly either zero, producing a linear reconstruction or a sigmoid. The parameters of the decoder are a bias vector and

a matrix, W' . The aim function for training an autoencoder is to identify parameters $\theta = (W, b_X, b_Y)$ that minimize the reconstruction loss on the provided dataset X .

$$\theta = \min_{\theta} L(X, X') = \min_{\theta} L(X, g(f(X))) \quad (2.6)$$

The reconstruction loss (L_2) in nonlinear reconstruction is often due to cross-entropy:

$$L_2(\theta) = - \sum_{i=1}^n [x_i \log(y_i) + (1 - x_i) \log(1 - y_i)] \quad (2.7)$$

where $x_i \in X$, $x_0 \in X_0$, and $y_i \in Y$.

Once the model is fitted, the reconstruction component may be deleted and the model up to the bottleneck can be utilized. At the bottleneck, the model outputs a fixed-length vector containing a compressed version of the input data. The model may then be fed data from the domain, and the model's output at the bottleneck can be utilized for dimensionality reduction [20].

2.3.3 Long Short Term Memory (LSTM)

The LSTM architecture is a variant of the RNN architecture, which is a recursive neural network technique suitable for modeling sequential data. An RNN's defining property is its network delay recursion, which allows it to accurately characterize the dynamic behavior of systems. Additionally, RNNs save a vector of activations for each time step, making the RNN a very deep neural network. However, training RNNs to learn long-term dependencies in time series data is often problematic because of the exploding and vanishing gradient. Both issues are created by the RNN's iterative nature, which produces a gradient that is basically equivalent to the recurrent weight matrix but multiplied by a large factor. These matrix powers cause the gradient to rise or contract in a proportionate manner to the number of time steps in the dataset.

Nonetheless, the exploding gradients issue has been proven to be very straightforward to solve using a method known as gradient clipping, which essentially trims gradients whose norms surpass a threshold. One benefit of the gradient clipping approach is that it maintains a small gradient for most of the learning period, while too small a gradient impairs learning or convergence. By comparison, the vanishing gradient issue is highly challenging since it reduces the gradient component of long-term relationships while increasing the gradient component of short-term dependencies. As a result, the RNN is adept at learning short-term dependencies but struggles with long-term dependencies.

The LSTM's architecture gives it an efficient solution to the RNN's vanishing gradient issue. It makes use of a memory cell that is capable of storing long-term dependencies in sequential data. The LSTM memory cell is comprised of four gates: an input gate, an output gate, a forget gate, and a self-recurrent neuron. These gates are in charge of regulating the interactions between memory units. More precisely, the input gate determines whether or not the input signal is capable of modifying the state of the memory cell. In comparison, the output gate determines whether it is permitted to affect

the state of other memory cells, whereas the forget gate has the option of forgetting or remembering its prior state. The diagram 2.2 illustrates the gates, concealed outputs, and cell states [21].

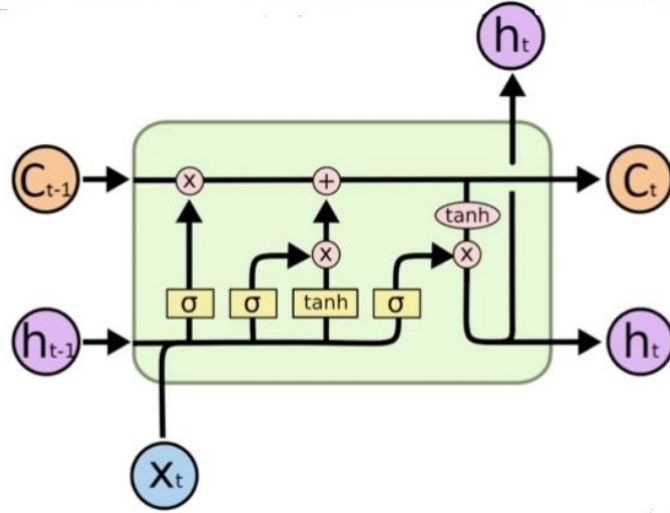


Figure 2.2: LSTM Cell Architecture

2.3.4 Generative Adversarial Network

A GAN is composed of two neural networks that compete in a min-max game: a discriminator and a generator. A discriminative network D is trained to differentiate between genuine and false data by being alternately presented with sequences from both classes and therefore adapted to give labels to actual and fake data. The rival generator network G generates synthetic data by sampling from a uniform or normal distribution. It then attempts to deceive D in each training phase by providing fictitious data alongside the true class label.

Thus, throughout the duration of training, the generator learns to produce sequences that D is unable to differentiate from actual data. Fitting the proposed networks such as LSTM in an adversarial environment enables the generation of an infinite number of time series data that is statistically comparable to actual data.

A GAN's training objective is to determine the probability distribution of the training data and to produce synthetic samples selected from it. As a result, the objective of the GAN is to learn the probability distribution of the real-world time series data and to create statistically equivalent time series data. The GAN's capacity to learn and reflect essential properties of actual data is heavily dependent on the quality of the training samples. A GAN is regarded as successful if it is capable of reproducing the distribution of the training data without being specifically instructed to do so.

If the actual probability distributions of the training samples are too varied, the training may fail, resulting in the GAN learning to produce samples from a single probability distribution while ignoring the others. This kind of failure is referred to as "mode collapse", and it occurs when the GAN fails to capture the variety of the training data [22].

There are various possible strategies for avoiding mode collapse in GANs, including the Wasserstein loss. It is claimed that wasserstein loss aids in mode collapse. However, maximizing the regularized wasserstein loss involves computing a second derivative of the loss function, which increases the execution cost and slows the training process [23].

2.3.5 Anomaly Detection using both Discrimination and Reconstruction

Two mapping functions are mentioned between the domains X and Z in the model, namely $E : X \rightarrow Z$ and $G : Z \rightarrow X$. X specifies the domain of the input data, which describes the provided training examples from the SWaT dataset.

Z is the latent domain, from which random vectors z are sampled to represent white noise. For simplicity, the notation x_i is used to signify a time series of length t beginning at time step i . The input time series is reconstructed using the mapping functions: $x_i \rightarrow E(x_i) \rightarrow G(E(x_i)) \approx x_i$.

The two mapping functions, E and G are used by using adversarial learning techniques. The two mapping functions are referred to as generators. E acts as an encoder, mapping the time series sequences to the latent space, while G acts as a decoder, converting the latent space to the reconstructed time series. Additionally, two antagonistic discriminators, C_x and C_z are introduced. C_x is used to discriminate between genuine time series sequences from X and generated time series sequences from $G(z)$, whilst C_z is used to quantify the mapping into latent space's performance. In other words, G is attempting to deceive C_x by producing sequences that seem to be genuine. Thus, the high-level objective is composed of two components [5]:

- Wasserstein loss, which is used to align the distribution of reconstructed time series sequences with the distribution of data in the target domain.
- cycle consistency loss, which is used to avoid contradictions between E and G .

Loss Functions

The mode collapse issue exists in the original formulation of GAN, which uses ordinary adversarial losses where C_x generates a probability score ranging from 0 to 1, representing the input time series sequence's truth. To be more precise, the generator only learns a tiny part of the data's variability, preventing it from completely converging to the desired distribution. This is mostly because the generator likes to generate samples that have previously been shown to be effective in fooling the critic and is hesitant to generate new ones, even if the new ones might be beneficial in capturing more "modes" in the data.

To circumvent this constraint, the GAN is trained using the wasserstein loss as the adversarial loss when training the critic network using the wasserstein-1 distance. Let P_X denote the distribution over X . It is obtained the following objective for the mapping function $G : Z \rightarrow X$ and its Critic C_x :

$$V_x(C_x, G) = \min_G \max_{C_x} E_{x \sim P_X} [C_x(x)] - E_{x \sim P_X} [C_x(G(x))] \quad (2.8)$$

where C_x indicates the set of continuous 1-Lipschitz functions. The Lipschitz continuous functions confine the function's upper bound, further smoothing it. As a result, when weights are adjusted using gradient descent techniques, they do not vary drastically.

This minimizes the possibility of gradient explosions and improves the stability and reliability of the model training. Additionally, a gradient penalty regularization term is introduced during training to impose the 1-Lipschitz constraint. This penalizes gradients that are not equal to 1.

Similarly, a wasserstein loss is established for the mapping function $E : X \rightarrow Z$ and its Critic C_z using a similar technique.

$$V_z(C_z, E) = \min_E \max_{C_z} E_{z \sim P_Z}[C_z(z)] - E_{z \sim P_Z}[C_z(E(z))] \quad (2.9)$$

The second Critic C_z is used to differentiate between latent random samples $z \sim P_Z$ and encoded samples $E(x)$ with $x \sim P_X$.

The GAN is designed to rebuild the following time series: $x_i \rightarrow E(x_i) \rightarrow G(E(x_i)) \approx x_i$. However, training the GAN using adversarial losses such as wasserstein loss alone cannot ensure that each input x_i is mapped to a desirable output z_i , which is then mapped back to x_i . To minimize the search space for viable mapping functions, cycle consistency loss is modified for time series reconstruction, training the generative networks E and G with the modified cycle consistency loss by minimizing the L2 norm of the difference between the original and rebuilt samples:

$$V_{L2}(E, G) = E_{x \sim P_X}[\|G(E(x))\|_2] \quad (2.10)$$

Given the objective is anomaly identification, the L2 norm is employed rather than the L1 norm to highlight the significance of outlier data. It was also discovered that introducing the backward consistency loss had no effect on performance.

When all of the goals in 2.8, 2.9, and 2.10 are combined, the final MinMax problem is as follows:

$$\min_{E, G} \max_{C_x, C_z} V_x(C_x, G) + V_z(C_z, E) + V_{L2}(E, G) \quad (2.11)$$

There are two advantages to this design in terms of anomaly detection. To begin with, critic C_x has been trained to discriminate between genuine and fabricated time series sequences; hence, the critic's score may be used directly as an anomaly measure. Second, by encrypting and decoding a time series sequence using the two generators trained with cycle consistency loss, the difference between the original and decoded sequences may be utilized as a secondary metric of anomaly detection where the architecture is indicated in figure 2.3.

To get the training samples, the original time series data is partitioned into N sub-sequences using a sliding window with a window size of t and a step size of s . In practice, determining the ground truth is challenging because abnormal data items are not common. As a result, all of the training sample points are assumed to be normal. Additionally, time series data is constructed from random space using a normal distribution to denote the latent space's dimension. Following that, X and Z are given as inputs into the GAN model and trained with the aim given in 2.11. The anomaly scores or likelihoods may be calculated at each time step using the trained model and the critic output.

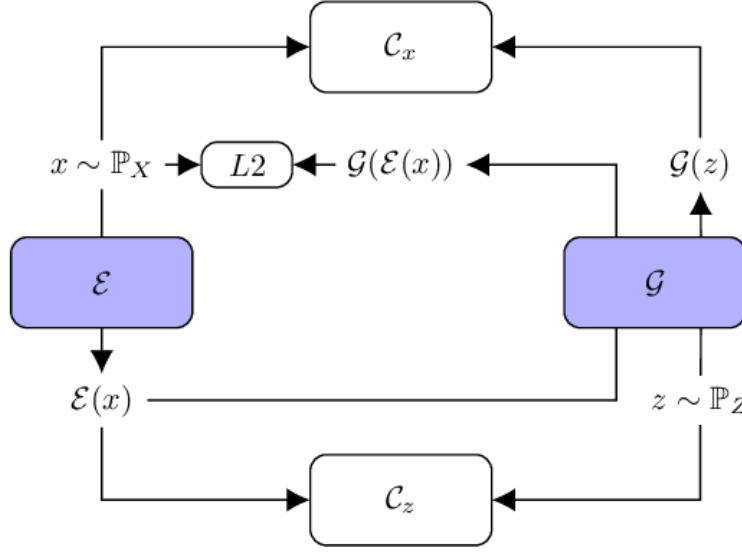


Figure 2.3: TadGAN Architecture

Given a sequence $x_i^{1 \dots t}$ of length t , hence referred to as x_i , a reconstructed sequence of the same length is created: $x_i \rightarrow E(x_i) \rightarrow G(E(x_i)) \approx x_i$. As a result, a collection of reconstructed values is obtained for each time point j .

As the final reconstructed value $\hat{x}_j$, the collection's median is used. It was also discovered that utilizing the median outperformed utilizing the mean. Now, the time series reconstruction has become $(\hat{x}^1, \hat{x}^2, \hat{x}^3, \dots, \hat{x}^T)$. To compute the reconstruction error, time dilation in real time (DTW) is used, which is also proposed as the best technique by TadGAN.

DTW is used to determine the degree of similarity between nearby locations. Its objective is to find the optimum match between two provided time sequences. Given two time series, $X = (x_{t-1}, x_{t-l+1}, \dots, x_{t+l})$ and $\hat{X} = (\hat{x}_{t-1}, \hat{x}_{t-l+1}, \dots, \hat{x}_{t+l})$, the warp route $W = (w_1, w_2, \dots, w_K)$ is identified, which specifies the shortest distance between the two curves subjected to start and end boundary requirements as well as limitations on continuity and monotonicity. The DTW distance between two time series, X and $\hat{X}$, is defined as follows:

$$s_t = W^* = DTW(X, \hat{X}) = \min_W \frac{1}{K} \sqrt{\sum_{k=1}^K w_k} \quad (2.12)$$

DTW, like area difference, is capable of identifying areas with tiny differences over a lengthy period of time, but it also handles temporal shift concerns.

Throughout the training phase, the critic C_x must discriminate between actual and synthetic input sequences. Because C_x is trained using the wasserstein-1 distance, the outputs may be interpreted as a measure of how genuine or fraudulent a sequence is.

As a result, once trained, the critic may be used directly as an anomalous measure for time series sequences.

As with the reconstruction errors, a collection of critic scores is also obtained at time step j . Kernel density estimation (KDE) is applied to the collection and then the largest value is picked as the smoothed value, c_j . It is found that the critic does indeed give different ratings to anomalous areas than to typical parts. This enables the identification of abnormal locations via the use of thresholding methods.

$RE(x)$ and the Critic outputs $C_x(x)$ cannot be utilized interchangeably as anomaly scores. Intuitively, a bigger $RE(x)$ and a lower $C_x(x)$ suggest a greater number of anomalies. As a result, the mean and standard deviation of $RE(x)$ and $C_x(x)$ are first calculated, and then normalized using their respective z-scores $Z_{RE}(x)$ and $Z_{C_x}(x)$. Larger z-scores imply a greater level of abnormality.

Several strategies are investigated for using $Z_{RE}(x)$ and $Z_{C_x}(x)$ and three distinct forms of $Z_{RE}(x)$ and $Z_{C_x}(x)$ are examined. To begin with, convex combination is used to try to combine them into a single value $a(x)$:

$$a(x) = \alpha Z_{RE}(x) + (1 - \alpha) Z_{C_x}(x) \quad (2.13)$$

where α determines the relative relevance of the two terms ($\alpha = 0.5$ by default). Secondly, both scores are multiplied in order to highlight the high values:

$$a(x) = \alpha Z_{RE}(x) \odot Z_{C_x}(x) \quad (2.14)$$

where by default $\alpha = 1$. It is observed that both approaches provide very reliable anomaly scores.

To identify sequences that are out of the ordinary, locally adaptive thresholding is used. After obtaining anomaly scores at each time step, thresholding methods are applied to the resultant sequences. The thresholds are calculated using sliding windows and experimentally determine the window size and step size. This is advantageous for identifying contextual abnormalities whose context is often unclear. The sliding window size specifies how many past anomaly scores are used to establish the current threshold. A basic static threshold is employed as four standard deviations from the window's mean for each sliding window. Then, those points are classified as anomalous whose anomaly score is above the threshold.

While sliding windows might improve recall of abnormalities, they can also generate a large number of false positives. An anomaly trimming strategy is applied to minimize false positives. To begin with, each anomalous sequence is presented using the maximum anomaly score, resulting in a collection of maximum values. After sorting these numbers from highest to lowest, the percent reduction is calculated. When the first percent does not exceed a predefined threshold, which is set to 0.1 by default, all following sequences are reclassified as normal [5].

2.4 Dataset

2.4.1 SWaT

SWaT is a fully operating, downsized water treatment plant with a compact footprint that produces 5 gallons of double filtered water per minute. This testbed is modeled after the huge contemporary water treatment facilities found in cities. Its primary objective is to allow empirically verified research on the secure and safe design of CPS. SWaT consists of six primary processes that correlate with the facility's physical and control components.

SWaT is comprised of a multi-layer communication network, programmable logic controllers, human machine interfaces, a supervisory control and data acquisition workstation, and a historian. The SCADA system receives data from the sensors, and the historian records it for later examination.

SWaT consists of two networks. Level 1 is a star network that enables communication between the SCADA system and the six PLCs allocated to each operation. Level 0 is a ring network that connects sensors and actuators to their associated PLCs. Sensors, actuators, and PLCs are all connected through wired or wireless connections. Only network data gathered through wired connections was used in the data gathering procedure.

The data collection procedure took 11 days in total. SWaT operated continuously 24 hours a day for the whole 11-day period. SWaT was conducted without incident for the first seven days of the eleven-day period. Throughout the following four days, attacks were launched. On the testbed, cyberattacks were implemented that had a variety of objectives and lasted anything from a few minutes to an hour. Depending on the cyberattack scenario, the system was either allowed to return to its regular functioning condition before another attack was started, or successive attacks were initiated.

All tanks in SWaT were drained prior to data collection; that is, the data collection procedure begins with SWaT in an empty condition. This initialization procedure was judged important to guarantee that all tanks are filled with unfiltered, untreated water [24].

2.4.2 Cyber-Attacks

The system was attacked in a methodical manner. The attack model, which examines an attacker's intent space for a particular CPS, is employed. This attack model may be used to construct attack processes and functions that are specifically targeted at a certain CPS. In this situation, an attack model is implemented specifically for the SWaT testbed. It is assumed that an attacker was successful in initiating an attack. Assuming an attacker succeeds in initiating an attack, the number of viable attack scenarios is limitless. In a CPS, an attack point may be a physical device or a point of entry through the communications network linking sensors or actuators to the controllers (PLCs) and the SCADA system.

Cyberattacks are classified into four categories based on the attack points in each level. A Single Stage Single Location (SSSP): A Single Stage Single Point cyberattack is one that concentrates only on a single point inside a CPS. A Single Stage Multiple Point (SSMP) cyberattack: A Single Stage Multiple Point attack targets two or more attack locations inside a CPS but only one stage. A Multi-Stage Single Point (MSSP) cyberattack is identical to an SSMP attack, except that the SSMP attack is carried out in stages. Multi-Stage Multi-Point (MSMP): A Multi-Stage Multi-Point cyberattack is an SSMP strike that utilizes two or more phases.

Throughout the data gathering procedure, the number of attack points and locations for each cyberattack are determined. Then the attack is implemented according to the attack point, the start state, the kind of attack, the value of the chosen sensor data to be given to the PLC, and the desired effect.

Throughout the data gathering procedure, a total of 36 attacks were initiated. The length of the assault varies according to the kind of attack. A series of ten-minute attacks are carried out sequentially, with a ten-minute pause between each cyberattack. Certain cyberattacks are carried out by allowing the system to stabilize before launching a further attack. The length of system stabilization varies depending on the kind of cyberattack. Certain cyberattacks have a greater impact on the system's dynamics, requiring more time for the system to settle. Simpler cyberattacks, such as those that alter flow rates, stabilize more quickly. Additionally, certain cyberattacks might not instantly take effect [24].

3 Literature Review

Time series data appears in many applications, such as satellite telemetry, server metrics, energy consumption, and even pandemic statistics. As a result, detecting and analyzing time series data has become critical for preventing cyberattacks, particularly in critical use cases such as cyber intrusion fault detection and preventive maintenance.

3.1 TadGAN: Time Series Anomaly Detection Using Generative Adversarial Networks

In this paper, a targeted research problem is given a multivariate time series to find a set of anomalous time segments that have unusual values or do not follow the expected temporal patterns. They use an unsupervised machine learning approach because of a lack of previously identified anomalies to train and optimize a supervised learning model and a lack of knowledge about what is normal to make an anomaly stand out. They train the machine learning model by feeding it with raw data to learn patterns in time series. When the model learns enough knowledge from the data, it will be able to generate the expected time series. If the expected time series deviated largely from the actual ones, then this indicates an anomaly.

They use an LSTM-based GAN to reconstruct the time series. To calculate the anomaly score of a time segment, they make the assumption that the anomalous values should not be generated along with the normal values. This reconstruction-based method learns to capture the latent structures of the given time series data and then reconstructs the data using their latent representations. The model maps the original time segment to a low-dimensional space and then map it back to the original space, which is called the generator. The assumption is that the data is lost information when it is mapped to a lower dimensional space, and thereby cannot be effectively reconstructed. In addition, they train the discriminator D_x to distinguish between the generated and real time series. In addition to the original GAN structures, TadGAN introduces an encoder E and an additional discriminator D_z to distinguish between the encoded signals and a random noise. Encoder E maps the original time series to a low-dimensional space, and G reconstructs it back to the original space.

To train the two mapping functions mentioned above, they employ two different terms in the objective function. The wasserstein loss matches the distributions of the generated time series sequences to the data distributions in the target domain. Generator G generates samples from the latent space where criticX (C_x) distinguishes the generated and real samples using the wasserstein distance. Similarly, they introduce

the wasserstein loss for the mapping of the function E and its critics C_z to distinguish between random sample and the encoded samples. To guarantee the mapping of inputs x_i , and desired latent output z_i , which will be further mapped back to the $\hat{x}_i$, the cycle consistency loss is introduced as the second term to the objective function. It minimizes the L_2 norm of the difference between the original and the reconstructed samples.

To find the discrepancies between the real and expected time series data, the paper uses both the reconstruction error and the critic score. On one hand, the reconstruction error can be calculated in three different ways. Point-wise distance calculates the point-wise distance between the two signals from actual and expected time series data. The area difference calculates the area between the two curves, which are constructed by the actual and expected time series. Finally, the dynamic time warping computes the DTW distances and identifies the regions of small differences over a long period of time, handling the time shift issues. On the other hand, the critic score can be viewed as the confidence level of how sure the model is, given that the signal is real. It can be utilized to give weight to the error. They propose two different ways to combine the critic score with the reconstruction error. The first one is a convex combination of the critic's score and the reconstruction error. The second one is the multiplication of both scores to emphasize the high values.

Once the errors from each timestamp are obtained, these errors are used in the thresholding strategies to find the normal segments of the time series. Using the locally adaptive thresholding method for a given time window, the paper proposes using four standard deviations from the mean of the window as the window's threshold. Pursuing such a strategy is helpful for extracting contextual anomalies specific to a window. After defining suspected time windows using adaptive thresholding, the authors compare them with the known anomalies that were given in the data set or with the inputs from the expert supervision to validate the accuracy of the pipeline. When the detected anomalies overlap with the known ground truth anomalies, they are counted as true positives. When the detected anomalies overlap with the ground truth normal time series, then it is counted as a false positive. The algorithm is compared with prediction-based methods like LSTM, ARIMA, and HTM; reconstruction based methods like LSTM autoencoder, Dense AE, and MADGAN; and commercial solutions like DeepAR and MS Azure using 11 different data sets, including NASA, Yahoo, Numenta, Amazon, and Twitter. TadGAN outperforms all the above-mentioned baseline methods by having the highest average F1 score with 0.7 [5].

3.2 MAD-GAN: Multivariate Anomaly Detection for Time Series Data with Generative Adversarial Networks

In this paper, they propose an unsupervised learning model suitable for multivariate time series data that can overcome the limitations of insufficient labeled data, the application of statistical control on multivariate data, and the complexity of multivariate time series. Their solution offers a GAN model-based LSTM to take advantage of

temporal dependency and a new anomaly score calculation using the GAN model's generator and discriminator losses. Anomaly detection through the MADGAN model consists of four major parts. The first consists of input data for model training, and the second trains the MADGAN model through the configuration data. The third step calculates the loss to improve the anomaly score. Finally, the anomaly score is calculated based on the reconstructed time series.

First, they built the GAN's generator and discriminator as two Long-Short-Term Recurrent Neural Networks to handle the time-series data (LSTM-RNN). The generator (G) generates fake time series using sequences from a random latent space as inputs, then passes the generated sequence samples to the discriminator (D), which attempts to distinguish the generated (i.e., "fake") data sequences from the actual (i.e., "real") normal training data sequences. Rather than considering each data stream separately, the MAD-GAN framework evaluates the complete variable set simultaneously in order to incorporate latent interactions between variables into the models. Before discrimination, a sliding window approach is used to partition the multivariate time series into sub-sequences. As a feature extraction technique, PCA is employed to reduce the dimensions of the input time series data, keeping the variance of the data close to the original. Iterative adversarial training is used to create the generator and discriminator. The GAN-trained discriminator and generator are used in the anomaly detection process to produce a composite anomaly score based on discrimination and reconstruction.

After a sufficient number of iterations, the parameters of D and G are updated based on the outputs of D, allowing the discriminator to be trained to be as sensitive as possible to assign correct labels to both real and fake sequences, and the generator to be as smart as possible to fool the discriminator (i.e., to mislead D to assign real labels to fake sequences). The generator G will have captured the latent multivariate distributions of the training sequences and may be seen as an implicit model of the system at a normal state if it can create realistic samples. Simultaneously, the resultant discriminator D was trained to be able to discriminate false (i.e. abnormal) data from actual (i.e. normal) data with a high degree of sensitivity.

To evaluate the algorithm, they use well-known data sets called SWaT and WADI. 51 variables (sensor readings and actuator states) were monitored over 11 days in the SWaT data set. 496,800 samples were obtained from the raw data under normal working conditions (data acquired in the first 7 days), and 449,919 samples were collected after various cyber-attacks were put into the system. Similarly, in the WADI data set, 789,371 samples for 103 variables were gathered in the first 14 days under normal working conditions, and 172,801 samples were obtained in the last two days when various cyber-attacks were put into the system.

Overall, MAD-GAN surpassed the most widely used unsupervised detection algorithms. The sole disadvantage is that dealing with longer sub-sequences takes LSTM-RNN longer, while the model gets slower with a sub-sequence length greater than 200 [2].

3.3 A GAN-based Anomaly Detection Approach for Imbalanced Industrial Time Series

This article presents a unique technique for detecting anomalies that is based on generative adversarial networks. Anomaly scores are calculated using both apparent and hidden loss. It identifies anomalies without any prior knowledge of the data by producing much higher anomaly scores when an abnormal sample is introduced into the trained model. When the labels for normal and abnormal are skewed, classifiers using machine learning techniques prioritize the accuracy of the majority classes at the expense of the minority classes.

They suggest a GAN-based solution to handle the issue in this paper, especially for the unbalanced industrial data. The suggested network needs just normal samples, making it a more realistic network than any currently available. The approach's network structure is composed of three components: a feature extractor, a generator, and a discriminator. To decrease the quantity of the training data and save computational time, the most representative features are explored using a feature extractor. Simultaneously, the decoder G attempts to reconstruct the time series data. The generator's loss function is composed of three components: fraud loss, apparent loss, and latent loss. The purpose of fraud loss is to persuade the discriminator to misclassify produced samples from the generator as genuine industrial samples. Apparent loss quantifies the distance between actual samples and produced samples with bottleneck characteristics encoded. Additionally, the latent loss is used to reduce the distance between the latent representations of actual samples and the encoded bottleneck characteristics of produced samples. Additionally, it calculates anomaly scores during testing by using apparent loss. The bigger the imbalance factor, the fewer anomalous samples there are in both the training and testing sets from which the various sorts of failures may be identified.

Their method is comparable to that of AnoGAN and bidirectional adversarial generative networks (BiGAN). The whole performance is then compared using the receiver operating characteristic's area under the curve (ROC). Using the rolling bearing data set, the suggested technique delivers good classification results even when the signals are excessively noisy [4].

3.4 Anomaly Detection with Generative Adversarial Networks for Multivariate Time Series

They propose an unsupervised GAN-based anomaly detection (GAN-AD) method for a complex multi-process CPS with multiple networked sensors and actuators. The generator and discriminator are built as two LSTM-RNN, as in a typical GAN. They generate fake samples from a specific latent space and pass them to the discriminator, which tries to distinguish them from real samples. G can be viewed as an implicit model of the CPS, while D is a machine-learned discriminator able to discriminate between real and fake samples with high sensitivity. They propose exploiting both G and D for the

anomaly detection task by exploiting the residuals between real-time testing samples and reconstructed samples.

The generator G , a LSTM-RNN model, defines a probability distribution for the generated samples, which can be written as $G(z)$, where z is a distribution from the random latent space. The discriminator D is then trained to minimise the average negative cross-entropy between its predictions and sequence labels. Anomaly detection for CPSs time series data consists of the following two parts: The trained discriminator D (after a sufficient number of iterations of adversarial training) is a direct tool for anomaly detection. The generator G , which is capable of generating realistic samples, is actually a mapping from the latent space to real data space. That is to say, residuals between X and G could be utilized to identify anomalies in data. To find the optimal Z that corresponds to the testing samples, they first sample a random set of Z from the latent space and obtain reconstructed raw samples.

In this paper, they adopt the principal component analysis to project the data from the SWaT data set into a PC projection space before feeding it to the model. The Secure Water Treatment System is an operational test-bed for water treatment that represents a small-scale version of a large modern water treatment plant found in large cities. Since the SWaT data is recorded every second, the window length is set to 120. The LSTM network for the discriminator is relatively simpler, with 100 hidden units and a depth of 1. It is found that a higher latent space dimension generates better samples, especially when generating multivariate sequences. They observe that using more principal components could reduce false negatives due to the noise information brought in by adding less important PC dimensions. This observation demonstrates that multivariate detection is applicable for complex CPSs with interconnected IoT of sensors and actuators [6].

3.5 Adversarially Learned Anomaly Detection

They present a GAN-based anomaly detection approach (ALAD) in this study that is both successful and efficient in terms of test time. GANs, in their most basic form, are composed of two competing networks: a generator G and a discriminator D . Because each test case requires solving an optimization issue, this strategy is impracticable for huge data sets or real-time applications. A significant family of traditional anomaly detection algorithms is distance-based, assessing whether data is abnormal based on distances to its closest neighbors or clusters in the data. These techniques need an adequate distance metric for the data.

The ALAD approach provided here is most similar to the AnoGAN method. In contrast to AnoGAN, which employs a regular GAN, ALAD is based on bi-directional GANs and hence contains an encoder network for mapping data samples to latent variables. ALAD is able to skip the computationally intensive inference approach needed by AnoGAN by recovering the relevant latent variables using a single feed-forward pass through the encoder network at test time.

Spectral normalization of the discriminators was also shown to be effective when used

to regularize the encoder in the studies. Normal samples should be properly recreated, but abnormal samples will almost certainly be inaccurately reproduced and so must be decoded using a reconstruction-based anomaly detection technique. The anomaly score provided here is motivated by the feature-matching loss. It is based on the distance between samples in the cycle-consistency discriminator D 's feature space, as described by the layer before the logits. The discriminators' confidence is used to encode and reconstruct a sample correctly.

They assessed the ALAD technique using publicly accessible tabular and picture data sets such as KDDCup99, Arrhythmia, and CIFAR-10. The addition of the latent penalty improves the performance. ALAD is extremely competitive with and often outperforms state-of-the-art anomaly detection algorithms on a variety of tabular and picture data sets. During training, it learns an encoder from the data space to the latent space, making it substantially more efficient than AnoGAN at test time [3].

3.6 High-Performance Unsupervised Anomaly Detection for Cyber-Physical System Networks

This article employs a stacked denoising autoencoder to simulate the underlying distribution of industrial network packets. The network is composed of three layers: an input, an output, and a hidden layer. The combination of feature learning, unsupervised training, and parallelization enables the framework to be trained without the presence of aberrant network data.

They underline the need to maintain a lean and quick strategy in order to handle massive volumes of industrial network data in real time. When they concentrate on building a fast pipeline, they utilize hidden layers with fewer nodes than the input layer. This decreases memory and processing needs by storing and optimizing fewer weights. Buffering and batching packets to compensate for the time required to train the network enables effective data preparation and analysis. Anomaly detection may be applied to just a portion of the accessible network packets, and its correctness must be verified via packet-wise labeling.

They provide a semi-automatic method for labeling this sort of data and anticipate that by manually examining the batches with the largest reconstruction errors and their communication contexts, the traffic can be manually classified as normal or abnormal.

The SWaT data set is used to assess the model's detection skills and to apply quality controls. The data is divided into two sections: one that is free of packet-wise cyber attacks and another that contains attacks. They find that the SDA's feature learning step is capable of capturing sufficient data during training to identify these cyber attacks during the testing phase [1].

3.7 Anomaly Detection in Cyber Physical Systems using Recurrent Neural Networks

This article discusses an unsupervised technique for detecting cyber-attacks on cyber-physical systems. CPSs are physically linked systems that provide important functions such as water treatment and distribution that are vulnerable to cyber attacks. Not only does the suggested technique identify abnormalities in the CPS, but it also identifies the attacked sensor. Due to the fact that anomalies and cyber-attacks often occur over time, correlating time-series data provides us with information about an anomaly over time that may be utilized to more precisely identify it.

The paper proposes the use of a LSTM-RNN to anticipate the sequence of data for the purpose of anomaly detection in this study. This strategy is very beneficial in real-world CPS settings where anomalous behaviors are uncommon. The difficulty of learning complicated temporal sequences is tackled with LSTMs that include memory blocks that instruct the network when to discard prior memory states and when to update hidden states. Each memory block has an input, an output gate, and a forget gate that regulates the input gate's operation and the amount of information processed. CUSUM computes the cumulative sum of the sequence predictions over time, allowing for the detection of tiny variations over time and thereby lowering the number of false positives. The objective is to find any discrepancies between the actual sensor data and the outputs of the trained model that forecasts the ideal sensor value under normal behavior.

The data set utilized to test their strategy comprises seven days of continuous operation under normal conditions and 496,800 samples used to simulate the sensors. Nine out of ten cyber attacks were identified in total. Additionally, the suggested technique proved effective in detecting SSMP attacks [25].

3.8 State-Aware Anomaly Detection for Industrial Control Systems

They present a state-aware anomaly detection technique that makes use of state-dependent detection thresholds and imposes stricter limitations on an attacker attempting to manipulate the process. The HAMIDS (Hierarchical Monitoring Intrusion Detection System) framework was created for the purpose of conducting security analysis and research in the area of Industrial Control Systems. This article examines industrial network protocols and gives a device-level understanding of industrial applications. It serves as a platform for more researchers to do industry research.

The CUSUM technique was developed for stateful aggregation and anomaly detection using residuals. When the CUSUM exceeds a threshold, the anomaly detector will sound an alert. An attacker may be able to alter sensor readings in order to shift the system's status to harmful circumstances in order to cause damage to the system or reduce the system's overall efficiency. A detection system may employ data gathered directly by sensors or may get processed data via PLCs or SCADA systems.

The authors are particularly interested in the water tank level and the state of the industrial control system's pumps and valves from the SWaT dataset. The attacker targets certain stages in the process that have a high or low residual value. As a result, neither stateless nor stateful detection approaches are capable of detecting cyber attacks. The proposed detector identifies these types of attacks whenever they exceed a predefined CUSUM threshold. In comparison to current state-of-the-art stateful detection systems, the suggested approach generates fewer false alarms.

Various scenarios, such as detecting coordinated injection cyber attacks, were evaluated using conventional non-state-aware stateful detectors. This research presents a framework for state-aware anomaly detection in industrial control systems. The framework is built by adding functionality for recording, processing, analysis, and anomaly detection to the HAMIDS framework. Their experimental findings reveal that the suggested strategy outperforms previous work by requiring less exposure time and allowing for improved detection [26].

3.9 Fence GAN: Towards Better Anomaly Detection

The conventional GAN-based anomaly detection algorithms promote sample distribution overlap with the actual data. However, the resultant discriminator is ineffective at identifying unusual data. They suggest a straightforward change to the GAN aim that results in produced samples that sit on the edge of the data rather than overlap it. The proposed technique, Fence GAN (FGAN), trains with the modified goal in the conventional adversarial way and may be utilized as an anomaly detector.

The generator's loss function is specified in FGAN as the weighted sum of encirclement and dispersion losses, which optimizes the distance between produced data points and their center of mass. There is no assurance that the produced points will include all of X ; they may just encompass a portion of it. As with a normal GAN, the Fence GAN is built from a generator and a discriminator that are trained sequentially.

FGAN is evaluated for its ability to identify anomalies in three distinct datasets: MNIST, CIFAR10, and KDD99. The encirclement loss, dispersion loss, and discriminator loss in FGAN all contribute to the intended outcome of the produced samples, creating a tight border around the dataset when the hyperparameters are optimized.

The discriminator score is used as the anomaly score in Fence GAN. Unlike Efficient GAN and GANomaly, it does not depend on generator reconstruction loss and does not require alterations to the core GAN architecture [27].

3.10 Anomaly Detection for a Water Treatment System Using Unsupervised Machine Learning

They examine two approaches in this paper: one-class Support Vector Machines and Deep Neural Networks (DNN) tailored to time series data produced by a CPS (SVM).

The suggested DNNs calculate the probability distribution of the next status for each time step directly. SWaT, a testbed at the Singapore University of Technology and Design to promote cyber-security research, is the CPS on which they assess the learning architecture. Sensor data and actuator signals were modified before reaching the PLCs, pumps, and valves, thanks to the hijacking of data packets, and they are used to train the model. 26 of the 36 cyber attacks are single-state single-point attacks, with the remaining multi-stage attacks being more complicated.

They build a neural network to represent an LSTM, which accepts log entries up to the i -th position and converts them into probabilities for each of the five sensors. The output layer uses a bilinear function to combine the real actuator position in the i -th log entry with the value of the concealed layer. The outlier factor is calculated by adding together the log probabilities of the actuator locations and sensor readings.

They extract all the windows from a log and send them to an algorithm that identifies each window as normal or abnormal during the training phase. However, to identify outliers, a threshold for outlier variables is required, which is difficult to achieve theoretically. When testing data is normalized using the mean and variance of the training data, DNN outperforms the commonly used one-class SVM algorithm. This might imply that changes in long-term patterns are more relevant to SVM than absolute values.

The DNN and SVM detectors are both capable of detecting abnormalities at similar rates, but the SVM detector is more prone to false alarms. Overall, DNN outperforms SVM in identifying anomalies when sudden changes in certain sensor data may be the source of false positives in the SVM [28].

3.11 DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning

Log analysis is an effort to develop a sequential model for free-text log data that is unstructured. DeepLog, an unsupervised streaming parser that parses incoming log entries in real time, is the state-of-the-art log parsing approach. The training step uses just a small percentage of the underlying system's log records.

A window of log keys is used as an input in DeepLog, and the log key value is used as an output. The dimensionality of the hidden state vector determines the capacity of an LSTM block. They train an anomaly detection model using this method by treating each parameter value vector sequence as an independent time series, as if it were a log key.

The time elapsed between successive log entries is stored in each parameter value vector by DeepLog. In this time series, it may discover unexpected patterns in one or more dimensions. A performance anomaly, for example, may be interpreted as a "slow down." A job like VM generation creates a succession of log entries, whereas each log key represents the execution of a log printing line in the source code. To record the execution route of every job, DeepLog may create a workflow model as a finite state

automaton.

Concurrency is available inside each job, and a log key may occur in many tasks. The purpose is to create a workflow model for each activity based on its log key sequence by separating log entries for various tasks in a log. A log key value from a set of various keys is to be followed by a log sequence. For each process phase, the chances of these keys appearing add up to one. They regard a task as an "if-else" branch of the job if it contains relatively few log keys and always appears alongside another task. This enables the creation of process models by creating a co-occurrence matrix for each distance value. The objective is to produce a collection of tasks [29].

3.12 Detecting Cyberattacks in Industrial Control Systems Using Convolutional Neural Networks

The SWaT dataset is used to evaluate the performance of a long-short term memory LSTM-based deep neural network with a one-class support vector machine in detecting cyber attacks on all stages of the same process. They demonstrate in this study that 1D CNNs can be utilized effectively to identify cyberattacks in complicated multivariant ICS data. They identify anomalous behavior windows based on the departure of prediction error from observed statistics above the specified threshold for at least the specified time frame.

They conducted experiments using recurrent neural networks, a prominent kind of neural network design for time series data processing. They predicted the time frame immediately after the input, which was similar to inputting the first 200 seconds of data to predict the next 200 seconds. This trend is repeated throughout the assessments.

They also experimented with gated recurrent cells (GRU), bidirectional recurrent neural network design, and encoder-decoder LSTMs. Additionally, they examined convolutional neural networks, more precisely 1D CNNs. Inception layers are well-known for delivering exceptional performance at a minimal computational cost. They implemented ResNet as a reference implementation. The tests featured a vast parameter space, which included the length of the input sequence, the size of the network's internal state, the length of the output sequence, the lag between the input and output, the depth of the network, the dropout value, and the type (variational vs. regular).

After a small number of training epochs, all neural networks achieved an RMSE of 0.02 or less. The training error rate for LSTMs and inception-based convolutional networks is the lowest. Other combinations are not far behind, and with further iterations, they give comparable outcomes. Although an eight-layer convolutional network produced the greatest results, a four-layer network was not far behind [30].

3.13 Multi-level Anomaly Detection in Industrial Control Systems via Package Signatures and LSTM networks

The authors have established a methodology for rapidly developing an ICS-specific IDS with little human involvement. The framework detects anomalies at both the content and time-series level.

It is designed to aid in the detection of sophisticated persistent cyberattacks and collective abnormalities. They apply their paradigm to a publicly available dataset derived from a supervisory control and data acquisition system for a gas-fired power station. A sequence of functions constructs the signature for a network package. $g(x)$ is a generating function that assigns a distinct value to each unique combination of its arguments and concatenates them into a string using a special character.

A Bloom filter is a probabilistic data structure that is used to determine whether an element belongs to a set. Due to possible memory constraints, they use a bloom filter to effectively store the signature database for regular network packages and then identify abnormalities. They use a bloom filter as a lightweight and quick package-level anomaly detector.

They propose a layered LSTM neural network for detecting anomalies at the time series level. Forget gates may be used to reset memory cells, allowing them to retain important information while discarding obsolete data. LSTM networks built from these memory cells may be used to represent multivariate time series sequences without the necessity for a preset fixed-size time frame. Each layer outputs the chance that the following package's signature will have a certain value given the time series input. Each feature is a single hot encoded unit, and each unit is sent through the model's several layers. After that, the model is trained to minimize this indicator function, referred to as top-k calibration.

They provided a framework that combines anomaly detection models at the package and time-series level. This combined framework offers an additional feature that may be regarded as supplementary data that informs the model whether or not a certain package is an anomaly. Each package does not need to send its signature to the time-series level detector since it is not even in the signature database. The detector's false positive rate may be tightly controlled by adjusting the granularity of feature discretization.

The authors demonstrate that the mixed LSTM network model is resistant to noisy inputs such as packages and sensor readings from ICS networks. Both models' memory decreases rapidly when the number of anomalous packages that are close to regular packages increases [31].

3.14 Time Series Analysis: Unsupervised Anomaly Detection Beyond Outlier Detection

Cluster Evolution is a technique for detecting anomalies that combines cluster evolution (CE) with time series analysis (TSA). The method reads log lines from either a file or a

stream. This phase removes the time stamps associated with each log line since they are unnecessary for clustering. When log lines are clustered over time periods, a succession of cluster maps is produced. These maps are used to calculate metrics that indicate the presence of abnormal behavior.

The models are then utilized to anticipate a future value and a one-step forward prediction interval. Analysis enables us to determine if time series that have previously correlated cease to do so or whether new correlations between clusters develop, both of which are indicative of an anomaly. Not only is each log line clustered once to build the cluster map for the time window in which it originated, but it is also assigned to the prior and subsequent time windows' cluster maps.

If a cluster decreases in size between time steps, it will rise in size; otherwise, it will stay constant in size. These measures account for both the consequences of cluster transitions and the evolutionary links between them. A time series is a collection of values connected with certain moments in time. They are modeled using autoregressive integrated moving-average processes. Extrapolating the model into the future enables the forecasting of the value immediately after the previous known value.

The aggregated anomaly score for all evolving clusters that have existed for at least 20 time steps. Over 300 evolving clusters were found, accounting for more than 90% of all log lines. This technique benefits from the fact that each of the various developing clusters is specialized for certain log line kinds.

Satisfactory results are obtained with temporal window widths comparable to cyber attack durations (10–30 minutes). Due to the coarse texture of large time frames, abnormalities are readily missed. Smaller time windows provide greater granularity and decrease the average response time, such as the period between the occurrence of an anomaly (time window 2) and its detection. At high similarity levels, precision scores of between 0.2 and 0.3 are achieved. The benefit of moderate tube diameters becomes obvious only when accuracy and recall are coupled. However, the model is harmed by data poisoning, such as abnormal behavior that impairs future detections [32].

3.15 EDMAND: Edge-Based Multi-Level Anomaly Detection for SCADA Networks

Between remote devices and the broad area SCADA network, EDMAND is implemented at each substation. EDMAND continuously monitors all passing traffic and notifies control centers when anomalies are detected. SCADA data may be classified into three tiers, each with its own unique set of properties. EDMAND is made up of three primary components: a data extractor, an anomaly detector, and an alert manager. The data extractor is accomplished via the use of Bro, a network security monitor.

The anomaly detector is divided into three levels, each of which employs a unique method for detecting anomalies. The first type occurs when data with a new index field is encountered with an anomaly score of 1. The second kind is aberrant values in data fields, which examine frequency and current data using a bayesian technique.

Analog measures like frequency, voltage, current, and power are quite prevalent. They classify them and discover abnormalities using the relevant procedures for each type. For frequency and voltage, means and standard deviation are employed.

Two components comprise the alert manager: the alert aggregator and the alert scheduler. The following fields are similar to all alerts issued by various processors: index field, alert type, timestamp, confidence score, statistical fields, and aberrant data [33].

3.16 Neural Contextual Anomaly Detection for Time Series

They provide a system for detecting anomalies in time series that scales cleanly from unsupervised to supervised settings, in this study. Time series anomaly identification is based on the division of each time series into overlapping, fixed-size windows. Each window is subdivided further into two sections: a context window and a suspect window. Temporal convolutional networks are used to translate these windows into neural representations (embedding). The objective is to forecast anomaly labels. Their technique is founded on the notion that by comparing representation vectors, they may detect abnormalities.

They use a neural network feature extractor that has been trained in such a manner that representations are pushed together in the absence of an abnormality in the suspect window/s and apart in the absence of an anomaly. The model is trained to reward each occurrence of an abnormality in the suspect window with a high score. Each time point may then be assigned numerous anomaly ratings and assigned to distinct suspect windows corresponding to different rolling windows.

MIXUP may help the model generalize even when there is a big discrepancy between injected and real abnormalities. This data augmentation strategy increases the diversity of training instances, but more crucially, the soft labels provide smoother, more generalizable decision functions. They train and assess a single model on all the training time series in each dataset. They use brief suspect windows that enable them to determine whether or not a location is unusual upon first observation.

They developed techniques for injecting anomalies that replicate the genuine abnormalities in the SMAP dataset. When only COE and po are used, it is more difficult to find for the model because they cannot provide comparable behavior. NCAD is a state-of-the-art approach for detecting anomalies in time data. They show that when combined with data augmentation techniques, expressive neural representations for time series may outperform more conventional approaches such as prediction models or methods based on reconstruction error [34].

4 Methodology

The pipeline consists of six stages:

- Data preprocessing
- Feature selection/feature extraction
- Data preparation
- Dynamic construction of the neural network architecture
- GAN training and reconstruction of the test data
- Anomaly score computation

As mentioned earlier, preprocessing is one of the most important stages for increasing the performance of the neural network. After preprocessing, various feature extraction techniques are applied to the data to observe their effects after training and eventually on the anomaly scores. Before the training, since these feature extraction techniques alter the shape of the data, the neural network hyperparameters are set accordingly. After training, the model is fed testing data to reconstruct the data without the anomalies. Anomaly scores for each time window are calculated and fed to the adaptive thresholding algorithm. Finally, the points above the threshold are identified as anomalies. Trying multiple feature selection and feature extraction algorithms is aimed to provide with insights into the dataset as well as ways of improving the GAN's performance. The complete pipeline is shown in the figure 4.1

At the beginning, all the algorithms are implemented using a Jupyter notebook to visualise and understand the dataset and code execution easily. Afterwards, the whole pipeline is implemented using Python for more efficient execution for grid search and multiprocessing.

4.1 Data Preprocessing

4.1.1 SWaT Dataset

Within the context of data preparation, understanding the dataset is the most important step to being able to apply the right techniques. The SWaT dataset contains 53 columns, including timestamp and Normal/Attack which indicate the timestamp of the recordings of the sensors and the label indicating whether these sensors are influenced by an attack, respectively. The timestamp column is in the format of *dd/mm/yyyy hh:mm:ss* which

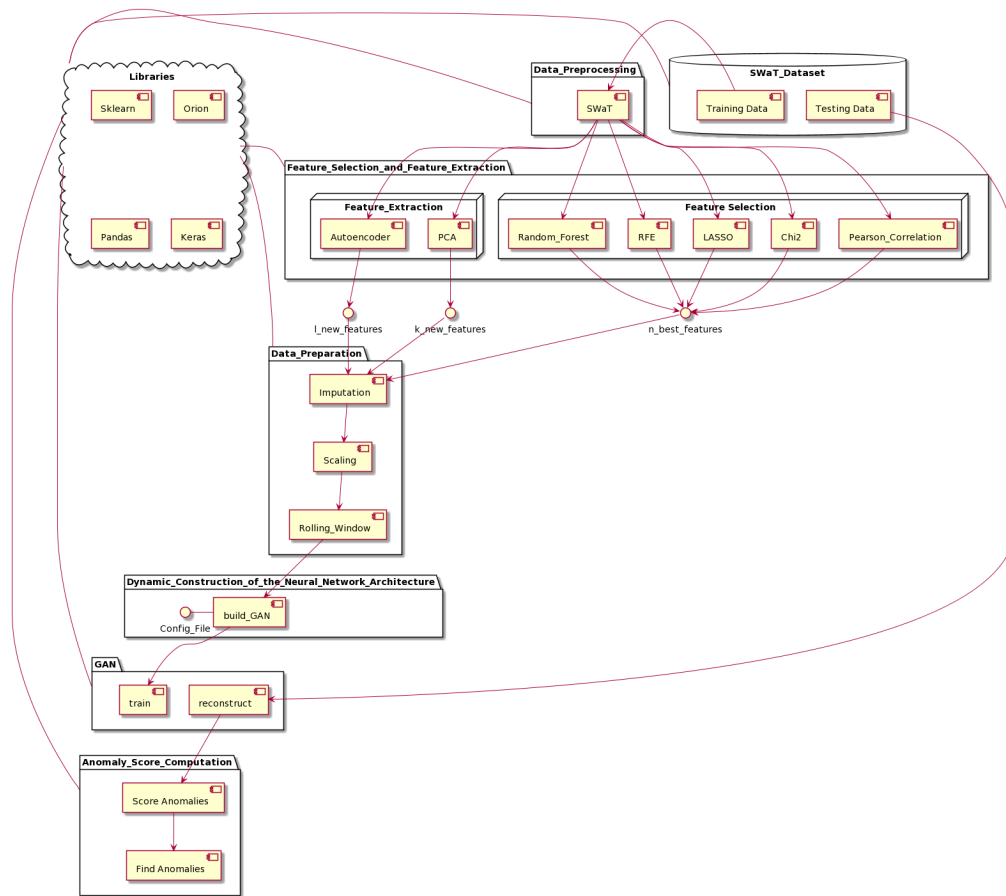


Figure 4.1: GAN Pipeline

is converted into seconds to separate into time windows easily and to increase the performance of the GAN.

The dataset is read using the `read_csv` method from the pandas¹ library, which takes the path of the file as an argument. After reading the csv, the data is registered into a DataFrame type variable. It is a data structure that also contains labeled axes like columns and rows. It is similar to the dictionary data type containing series objects where arithmetic operations align on both row and column labels. First, the names of the columns are corrected because the timestamp column was written with a space in front of it, generating a Not Found KeyError.

The first 21,600 rows of the training data are discarded because of the initialization of the plant. After 21600 rows, the plant was functioning in the normal order, which formed the normal dataset for the training. This can be observed in the figure 4.2. For testing, the first $\frac{7}{8}$ of the dataset is used due to the abnormality of the data at the end. This abnormality can be observed in the figure 4.3 indicating the attack complete data and $\frac{7}{8}$ of the attack data [35].

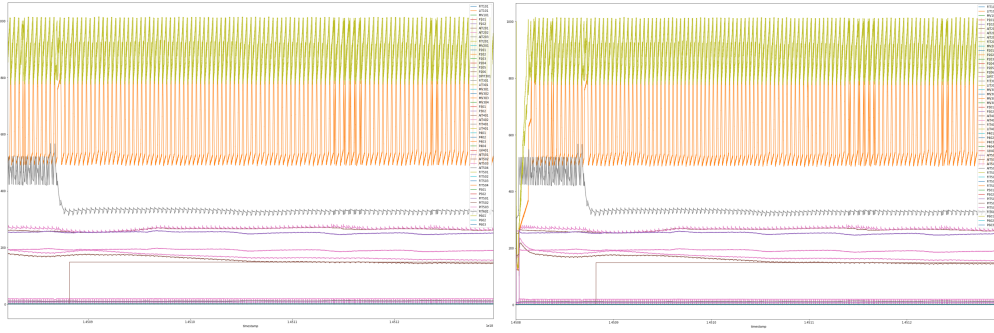


Figure 4.2: SWaT Normal Dataset Comparison

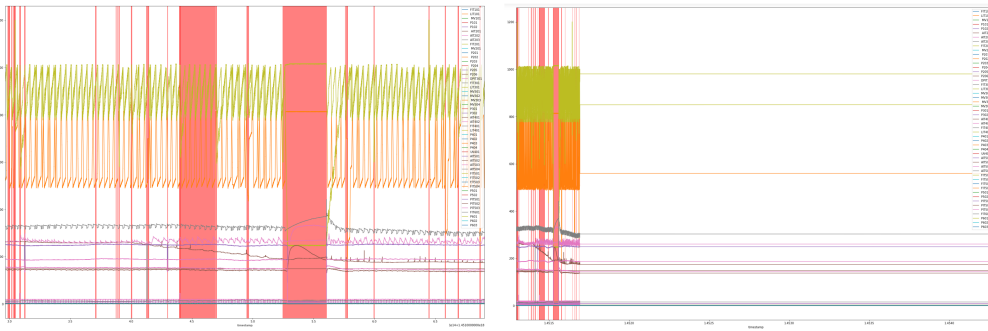


Figure 4.3: SWaT Attack Dataset Comparison

¹<https://pandas.pydata.org/>

4.2 Feature Selection and Extraction

4.2.1 Feature Selection

Five different feature selection techniques are employed to find the least and most effective features. Each technique chooses some features from 51 options. These features and the points gathered are displayed in a tabular format. The features that gather more points than 3 are selected as the most important features that need to be preserved.

This table is formed using a DataFrame object from the Pandas library. The columns of the table are named after the feature selection techniques, where the last column, *Total* indicates the number of total points collected from each feature selection technique. The rows are sorted according to the *Total* column in descending order to visualize the most important features on top of the table.

Before moving on to how the results from each of the feature selection techniques are calculated, note that figure 4.4 contains 2 tables of feature scores. The first one indicates the selected features from the test dataset where various attacks are implemented. All the features have equivalent points in the latter table because none of the features was able to influence the label (Normal/Attack column) of the time series. We will talk more about this on the properties of the table in the discussion.

Pearson Correlation Coefficient

Standardization of a dataset is a typical need for many machine learning estimators; they may exhibit undesirable behavior if the individual features do not resemble standard, normally distributed data [36].

StandardScaler is used as a preprocessing step before applying the Pearson correlation coefficient. It is a method from the *sklearn.preprocessing* library. It normalizes features by omitting the mean and scaling to unit variance.

The standard score for a sample x is determined as $z = (x - u)/s$, where u is the mean of the training samples and s denotes their standard deviation. By calculating the necessary statistics on the training set samples, centering and scaling are performed individually for each feature. The mean and standard deviation are then saved for use with subsequent data through the *transform* method.

Afterwards, each feature's correlation with y is calculated. By looping through the columns, the *corcoeff* function is called from the *numpy* library². The result is then stored in an array where the nan values are changed with 0s. Finally, the feature names that have a correlation with y are found and labeled as *True* otherwise *False*.

Chi-Squared

The Chi-square test is another feature selection technique investigated in this thesis. The data fed into the chi-square test needs to be non-negative. Therefore, using another

²<https://numpy.org/>

Out[11]:

	Feature	Pearson	Chi-2	RFE	Logistics	Random Forest	Total
1	PIT503	True	True	True	True	True	5
2	PIT501	True	True	True	True	True	5
3	LIT101	True	True	True	True	True	5
4	FIT504	True	True	True	True	True	5
5	FIT502	True	True	True	True	True	5
6	FIT501	True	True	True	True	True	5
7	FIT401	True	True	True	True	True	5
8	FIT201	True	True	True	True	True	5
9	DPIT301	True	True	True	True	True	5
10	AIT202	True	True	True	True	True	5
11	UV401	True	True	True	False	True	4
12	PIT502	True	True	True	False	True	4
13	P501	True	True	True	False	True	4
14	P403	True	True	True	True	False	4
15	P102	True	True	True	True	False	4
16	MV304	True	True	True	True	False	4
17	MV301	True	True	True	True	False	4
18	LIT301	True	True	True	False	True	4
19	FIT503	True	True	True	False	True	4
20	AIT504	True	True	True	True	False	4
21	AIT503	True	True	True	True	False	4
22	AIT502	True	True	True	False	True	4
23	AIT501	True	True	True	False	True	4
24	AIT402	True	True	True	False	True	4
25	AIT401	True	True	True	True	False	4
26	AIT203	True	True	True	True	False	4
27	P201	True	True	True	True	False	4
28	AIT201	True	True	True	False	True	4
29	P603	True	True	True	False	False	3
30	P602	True	True	True	False	False	3
31	P601	True	True	True	False	False	3
32	P502	True	True	True	False	False	3
33	P404	True	True	True	False	False	3

Out[11]:

	Feature	Pearson	Chi-2	Random Forest	Total
1	UV401	True	True	True	3
2	PIT503	True	True	True	3
3	PIT502	True	True	True	3
4	PIT501	True	True	True	3
5	P603	True	True	True	3
6	P602	True	True	True	3
7	P601	True	True	True	3
8	P502	True	True	True	3
9	P501	True	True	True	3
10	P404	True	True	True	3
11	P403	True	True	True	3
12	P402	True	True	True	3
13	P401	True	True	True	3
14	P302	True	True	True	3
15	P301	True	True	True	3
16	P206	True	True	True	3
17	P205	True	True	True	3
18	P204	True	True	True	3
19	P203	True	True	True	3
20	P202	True	True	True	3
21	P201	True	True	True	3
22	P102	True	True	True	3
23	P101	True	True	True	3
24	MV304	True	True	True	3
25	MV303	True	True	True	3
26	MV302	True	True	True	3
27	MV301	True	True	True	3
28	MV201	True	True	True	3
29	MV101	True	True	True	3
30	LIT401	True	True	True	3

Figure 4.4: Feature Selection Table

preprocessing, all the data is set normalized between $[0, 1]$. *SelectKBest* is a function from the Scikit library which takes a score function as a parameter and returns features according to the k highest score.

From the same library, the *chi2* function is used to compute chi-squared statistics between each non-negative feature and class. It measures the dependence between stochastic variables. This score is used to select the features with the highest values for the test chi-squared statistic from the dataset, which must contain only non-negative features such as booleans or frequencies relative to the classes.

After initializing the *SelectKBest* object with the *chi2* score function, the non-negative data and the target label are fed to the *fit* function. It runs the score function to find the appropriate features. To retrieve an array of the indices of the selected features, *get_support* function is used, which is an attribute of the trained estimator. It returns a boolean array of shape $[\text{input features}]$, in which an element is True if and only if its corresponding feature is selected for retention. This array will be useful to create the selected features table at the end [36].

Recursive Feature Elimination

As the third feature selection technique, recursive feature elimination, is used. Logistic regression is provided as an external estimator that weights information such as linear model coefficients. The objective of recursive feature elimination (RFE) is to pick features by examining fewer and smaller sets of characteristics in a recursive fashion. To begin with, the estimator is trained on a small number of features, and the significance of each feature is determined using either a particular attribute or callable. Then, the least significant characteristics in the current collection of features are trimmed. This technique is continued recursively on the trimmed set until the required number of features is obtained.

The non-negative and normalized dataset is fed to the *fit* function of the RFE object, which is created using logistic regression as an estimator. Logistic regression is employed with the default *lbfgs* solver for the multivariate case. The max iteration count had to be set to 1000 because the solver did not converge for the smaller values. Once the RFE estimator object is created, the norm of X and the label array y are given as arguments to the *fit* function, similar to the *chi2* test. The support of the estimator provides an array of boolean values which indicate the indices of the selected features [36].

Lasso

Lasso is one of the embedded feature selection techniques. Embedded techniques make use of algorithms with built-in feature selection mechanisms. For instance, Lasso and RF each have their own way of selecting features. Lasso regression uses the L1 norm as a regularizer. Unlike ridge regression, Lasso's norm regularizer drives parameters to

zero. The higher the value of α , the fewer features have a non-zero value [36]:

$$\alpha \sum_{i=1}^K |w_i| \quad (4.1)$$

LogisticRegression with the penalty term L1 is given as an argument to the *SelectFromModel* alongside with the max number of features to create the selector. Afterwards, the selector is given the normalized non-negative version of the dataset as well as labels using the *fit* method. As mentioned before, the selected features are retrieved using the embedded *get_support* method. Unlike the other 3 previous methods, Lasso was able to eliminate 8 features out of 51 features present in the SWaT dataset, resulting in 43 features.

Random Forest

A random forest is a meta estimator that fits a number of decision tree classifiers to different subsamples of the dataset and then utilizes averaging to increase predicted accuracy and avoid overfitting. If *bootstrap = True* (the default), the size of the subsample is determined by the *max_samples* argument; otherwise, the whole dataset is used to create each tree. The relevance of features is determined in each decision tree by examining the node impurity. In a random forest, the ultimate feature significance is the average of all decision tree feature importances.

Additionally, when dividing a node during tree building, the optimal split is determined using either all input features or a random subset of size-max features. The goal of these two randomness sources is to reduce the forest estimator's variance. Indeed, individual decision trees display a significant degree of diversity and are prone to overfit. Forests with randomly injected randomness provide decision trees with partially dissociated prediction errors. By averaging their projections, certain inaccuracies may be eliminated. By merging distinct trees, random forests minimize volatility, sometimes at the expense of a modest increase in bias. In reality, the variance decrease is often large, resulting in a model that is overall superior [36].

Similarly, *SelectFromModel* is used with *RandomForestClassifier* as an estimator from the sklearn library. After initializing the estimator object with 100 estimators, the dataset with the labels is given to the *fit* function as an argument. The dataset is normalized using the *StandardScaler* to increase the performance of the estimator.

4.2.2 Feature Extraction

By retaining a subset of the original features, feature selection eliminates extraneous or superfluous characteristics (variables) from the provided data collection. Feature extraction does the same thing, except instead of retaining a subset, it generates new features. In other words, the feature names are not preserved at the end of the feature extraction process, but new features are created to represent the characteristics of the former features. This is the critical distinction between feature selection and extrac-

tion. Both are capable of being supervised or unsupervised. Variance and correlation thresholds are two ways for selecting features [37].

In this thesis, two different unsupervised feature extraction techniques are tested: PCA and Autoencoder. PCA is a good example of the feature extraction techniques that make use of the variance of the dataset. On the other hand, autoencoders are neural networks that are trained to reconstruct their original inputs, preserving the information and reducing the dimension of the dataset at the same time.

PCA

Similar to feature selection techniques, *StandardScaler* is used for data normalization. With the resulting normalized data, *PCA* method from the *sklearn.decomposition* library is called. This method is used to reduce the dimensionality of the data linearly by applying singular value decomposition to project it to a lower dimensional space. Prior to applying the SVD, the input data is centered but not scaled for each feature. The solver of the algorithm is defined as "auto" by default. Therefore, a default policy is used to pick the solver based on the shape of the input data and the dimensions of the resulting data; if the input data exceeds 500x500 and the number of dimensions to extract is less than 80% of the data's lowest dimension, the more efficient "randomized" technique is enabled. Alternatively, the whole SVD is calculated and optionally truncated.

If the argument of the *PCA* is between 0 and 1, then the method optimizes the number of dimensions of the output such that the amount of variance is preserved as specified in the argument. If the argument is an integer, then PCA uses the argument as the dimension of the output and optimizes the variance. In this thesis, the PCA method is used in both ways with the variables 0.99 and 5, where the resultant output dimensions are 23 and 5, respectively. Since both test and training data need to be processed using PCA, the output dimension might not be equal at the end when using the variance as an argument. This might then cause inconsistency between the trained anomaly detector and the test dataset. For instance, when the GAN is trained using a dataset with 23 dimensions, it requires the test dataset, which is used in prediction, to have the same number of dimensions. Since the difference between training and test datasets can create such inconsistency, the dimensions of the training set are used as the basis, and the resulting dimensions are used when running PCA for the test dataset. As a result, two sets of training data and testing data with 5 and 23 dimensions are created to train and test the anomaly detector.

Autoencoder

An autoencoder is made up of two sub-models: an encoder and a decoder. The encoder compresses the input, while the decoder tries to reconstruct the input using the encoder's compressed version. The encoder model is kept, and the decoder is deleted upon training. The encoder may then be used as a data preparation approach to extract features from raw data for training another machine learning model.

The autoencoder model's design purposefully complicates this by limiting the architecture to a bottleneck at the model's midway, from where the input data is reconstructed. Once the model is fitted, the reconstruction component may be deleted, and the model up to the bottleneck can be utilized. At the bottleneck, the model outputs a fixed-length vector containing a compressed version of the input data. The model may then be fed data from the domain, and the model's output at the bottleneck can be utilized as a feature vector in a supervised learning model, for visualization, or for dimensionality reduction more broadly.

The encoder consists of two hidden layers, the first with twice the number of inputs (e.g., 104) and the second with half the number of inputs (52). Following that, the model's design is adjusted such that the bottleneck layer has half the number of nodes (e.g., 26).

The decoder is defined similarly, but in reverse. It has two hidden layers, the first with the same number of inputs as the bottleneck (for example, 26), and the second with double the number of inputs (e.g., 52). The output layer will contain the same number of nodes as columns in the input data and will output numeric values using a linear activation function. We can see in 4.5 the autoencoder model's layers to get a sense of how the data flows through the model.

These layers are implemented using Tensorflow.Keras' layers package. The dense layer supports the following operations: $output = activation(dot(input, kernel) + bias)$, where *activation* is the element-wise activation function supplied as the activation parameter, *kernel* is the layer's weight matrix, and *bias* is the layer's bias vector. Batch normalization and leaky ReLU activation are utilized to ensure the model learns effectively.

The dense layer is followed by the BatchNormalization layer. Batch normalization employs a transformation that keeps the mean of the output near to zero and the standard deviation of the output close to one. Notably, batch normalization behaves differently throughout the training and inference phases. The layer normalizes its output using the mean and standard deviation of the current batch of inputs during training when using *fit* function. That is, the layer yields $gamma * (batch - mean(batch)) / \sqrt{var(batch) + epsilon} + beta$ for each normalized channel where *epsilon* is a minuscule constant configurable as part of the constructor arguments, *gamma* is a learning scaling factor with a default value of 1 and *beta* is an uninitialized learning offset factor. During inference, that is, when *evaluate* or *predict* are used, the layer normalizes its output using a moving average of the mean and standard deviation of the batches observed during training. As a result, $gamma * (batch - moving_mean) / \sqrt{moving_var + epsilon} + beta$ is returned, where *moving_mean* and *moving_var* are non-trainable variables that are modified whenever the layer is invoked in training mode [38].

$$moving_mean = moving_mean * momentum + mean(batch) * (1 - momentum) \quad (4.2)$$

$$moving_var = moving_var * momentum + var(batch) * (1 - momentum) \quad (4.3)$$

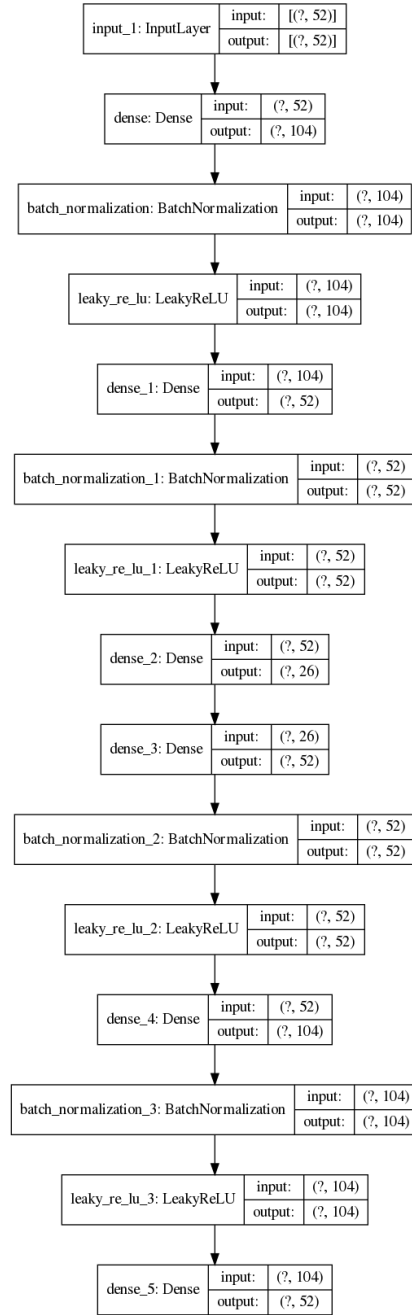


Figure 4.5: Autoencoder Architecture

As such, the layer will normalize its inputs during inference only after training on data with comparable statistics to the inference data.

LeakyReLU is used from the same `Keras.layers` library after the batch normalization layer. It allows small gradients when the units are not active:

$$\begin{aligned} f(x) &= \alpha x & \text{if } x < 0 \\ f(x) &= x & \text{if } x \geq 0 \end{aligned} \tag{4.4}$$

where α is 0.3 by default. On one hand, sparse activation is one of the advantages of using the ReLU. For example, only roughly 50% of hidden units are activated in a randomly started network or have a non-zero output. It also provides a more effective gradient propagation. There are fewer concerns with vanishing gradients than with sigmoidal activation functions that saturate in both directions. In addition to scale invariance, only comparison, addition, and multiplication are required for efficient computing. On the other hand, the ReLU function is not differentiable at 0, it is differentiable everywhere, and the value of the derivative at zero may be arbitrarily selected between 0 and 1. Other than the problems like not being centered on zero or unbounded, it also poses a dying ReLU issue. ReLU neurons may sometimes be driven into states where they are virtually dormant for all inputs. Due to no gradients traveling backward through the neuron in this condition, the neuron remains permanently inactive and "dies." This is a variant of the issue of disappearing gradients. In certain situations, a network's vast number of neurons might get locked in dead states, thus reducing the model's capacity. This issue often occurs when the learning rate is set to a value that is too high. That is why leaky ReLU is used instead, which assigns a modest positive slope, but performance is sacrificed.

Given that reconstruction is a form of multi-output regression issue, the model is fitted using the Adam variant of stochastic gradient descent that minimizes the mean squared error. Adam may be thought of as a synergistic mix of RMSprop and Stochastic Gradient Descent. It scales the learning rate using squared gradients, similar to RMSprop, and it makes use of momentum by utilizing the moving average of the gradient rather than the gradient itself, similar to SGD with momentum. Consider how it works in further detail.

Adam is a technique for calculating adaptive learning rates, which implies that it computes individual learning rates for various factors. The adaptive moment estimate is the source of its name. Adam adapts the learning rate for each weight of the neural network by estimating the first and second moments of gradient, where the N^{th} moment of a random variable is defined as its predicted value to the power of n .

In addition to the optimizer, the last argument to the `compile` function is the MSE as the loss function. The Mean Squared Error (MSE) is one of the simplest loss functions. The MSE is calculated by squaring the difference between your model's predictions and the ground truth and averaging them over the whole dataset. Because we are always squaring the losses, the MSE can never be negative. The MSE is mathematically defined as follows:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \tag{4.5}$$

Where N is the number of samples being compared. The MSE is excellent for ensuring that the trained model does not include any outlier predictions with big mistakes, since the MSE weights these errors more heavily owing to the squaring portion of the function. If the model produces a single really poor forecast, the function's squaring portion increases the mistake.

Following that, the model is trained for 200 epochs to recreate the input and monitor the model's performance on the dataset indicated in figure 4.6. At the end of the training, the following plot is drawn to show the learning curves for both the train and test sets to verify that the model correctly learnt the reconstruction issue.

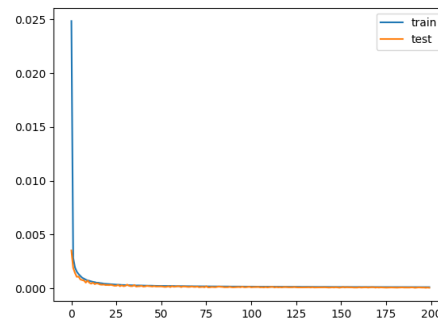


Figure 4.6: Autoencoder Loss Function

The important point is to only save the weights of the encoder since it is aimed to reduce the dimensions using the bottleneck layer. To do that, an encoder variable is created which has the dataset as the input and the bottleneck layer as the output. Finally, the weights of the encoder are stored in a *.h5* file to not train the model over and over again to use the predict method, but instead it is enough to load the weights.

4.3 Data Preparation

After either the feature selection or feature extraction techniques are applied, the resulting data is ready for a final preprocessing node before being fed to the GAN. First, both training and testing data are divided into indexes and values. *time_segments_aggregate* sorts the data first in ascending order by the timestamps. Then it aggregates the values of an interval. The interval value is also given as an argument to the method. Since 1 second is defined as 1,000,000,000, the interval value is set to 1,000,000,000. It returns at the end a numpy array of indexes and a numpy array of values containing the rest of the data.

Afterwards, *SimpleImputer* object is created from the *sklearn.impute* library to complete the missing values in the data. Numerous real-world datasets include missing values, which are often represented as blanks, NaNs, or other placeholders. However,

these datasets are incompatible with scikit-learn estimators, which presume that all values in an array are numerical and have and maintain meaning. A fundamental method for working with incomplete datasets is to delete whole rows and/or columns that contain missing values. However, this comes at the cost of potentially vital data being lost (even though incomplete). A more effective method is to impute missing values, or infer them from the known portion of the data. Once the imputer object is created, the *fit_transform* method is called which takes the resulting data as an input and returns the imputed data.

Standardization of datasets is a typical prerequisite for many deep learning estimation techniques; they will perform poorly if the constituent features do not resemble standard, normally distributed data. *MinMaxScaler* from the *sklearn.preprocessing* library is used to standardize the data between -1 and 1. Similar to the process in feature selection and extraction, first the scaler object is created using the *MinMaxScaler*. Then, both the training and testing data are given to the *fit_transform* method as an argument. The normalized characteristic of the resulting data is specifically during training the GAN since it lets the GAN converge faster.

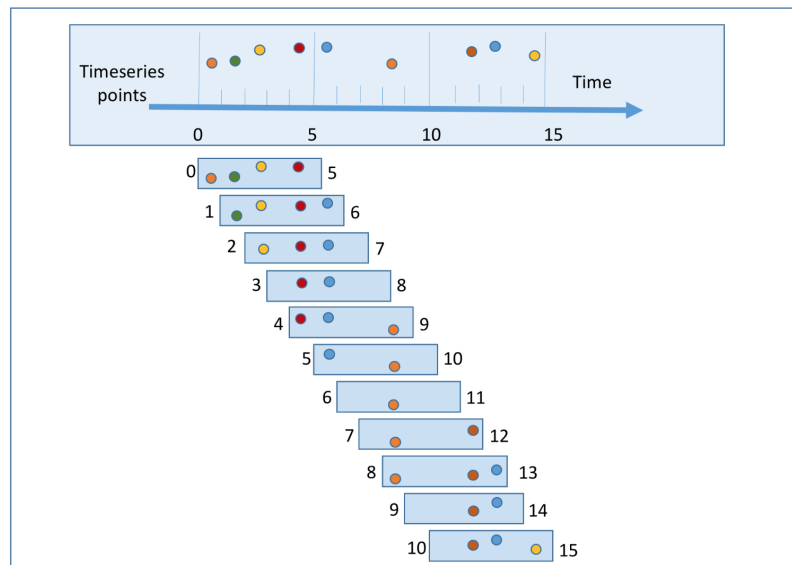


Figure 4.7: Rolling Window Schema

Finally, the last element of preprocessing is the *rolling_window_sequences*. It generates rolling window sequences from timeseries data. By rolling over the input sequence with a predefined window, the function generates an array of input sequences and an array of destination sequences. The function takes the training and testing data as input and returns the time sequence versions of the data along with their indexes. Then, it traverses the data, creating subsequences of the size given as an input. These subsequences are appended into the array to be returned. After this process moves to

the right, a predefined amount is given as an argument, *step_size* depicted in figure 4.7. Once the whole data set is traversed, the resulting window sequences of training and testing data are used in training and prediction methods, respectively.

4.4 Dynamic Construction of the Neural Network Architecture

A configuration file is used to setup the neural network architecture. *model.py* contains the hyperparameters as well as the layer configuration of the GAN. As mentioned before, the GAN can be divided into four components: an encoder, a generator, a discriminator for the encoder, and another discriminator for the generator. An array of dictionaries is used to define the architecture of the encoder. The input layer of the encoder contains an LSTM layer with 100 units, which defines the dimensionality of the output sequence, where the *return_sequences* is set to *False* to return the last output depicted as input_17 in figure 4.8. The LSTM layer is followed by a flattening layer that flattens the output. A dense layer, which has 20 units outputting a 20-dimension tensor, is defined as the last layer of the encoder network depicted as sequential_9 in 4.8. Therefore, the resulting latent output z has 20 dimensions.

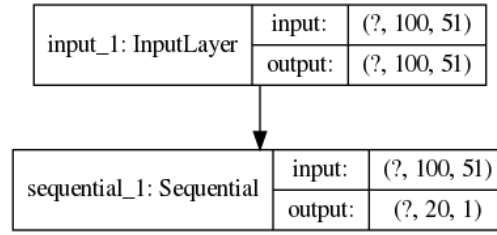


Figure 4.8: The Architecture of the Encoder

The encoder network is followed by the generator network. The first layer flattens the input of the generator. It is followed by a dense layer with 50 units, resulting in a tensor with 50 dimensions. The output of the dense layer is reshaped and fed to the bidirectional LSTM layer. The LSTM layer contains 64 units with a 0.2 dropout and a 0.2 recurrent dropout to allow for the linear transformation of the recurrent state. Next, a convolutional upsampling is used to increase the size by two, which is followed by another LSTM layer with 64 units and a dropout. *TimeDistributed* wraps a dense layer with 51 units to apply a layer to every temporal slice of the input gathered from the LSTM layer. Finally, the resulting tensor with 51 dimensions is inputted to the activation layer, which employs *tanh* as the activation function, mapping the input to the reconstructed values between -1 and 1 shown in figure 4.9.

Then, the discriminator for the reconstructed data x is introduced in the configuration file. *DiscriminatorX* checks if the data is reconstructed similar to the original data. Its first layer is a convolutional layer with 64 filters and a kernel size of 5. The output from the first layer is then passed to the activation layer where leaky ReLu is used with a slope

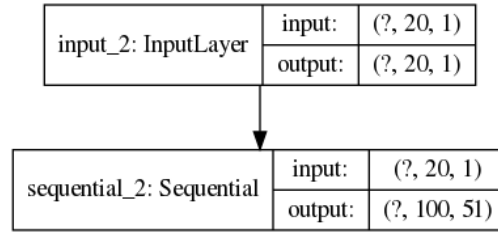


Figure 4.9: The Architecture of the Generator

of 0.2. As mentioned before, leaky ReLu is crucial to prevent all gradients converging to 0. The activation layer is followed by a dropout layer with a drop rate of 0.25. The same convolution activation and dropout layers are repeated three more times before the final flattening and dense layers. The final dense layer contains 51 neurons so that the dimensionality of the output matches the input dimensions, which is depicted in the figure 4.10. According to the preprocessing technique, the input dimensions may vary, so these hyperparameters are set during the program execution automatically.

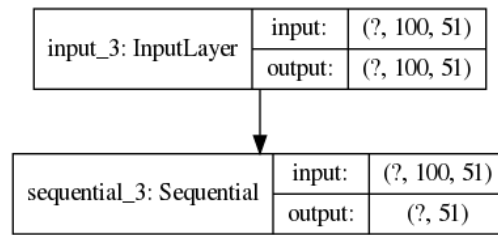


Figure 4.10: The Architecture of the Discriminator X

Unlike *DiscriminatorX*, dense layers are employed in sequential layers of *DiscriminatorZ* which controls how the encoder is able to predict latent variables z . The first layer of the network is the flatten layer used by the library Keras. It is followed by a dense layer containing 100 neurons, an activation layer, and a dropout layer sequentially. Leaky ReLu is employed in the activation layer where the slope α is set to 0.2. In the dropout layer, the dropout rate is set to 0.2, which helps prevent overfitting. This order of dense, activation, and dropout is stacked twice before the final dense layer with a single unit, which is depicted in the figure 4.11.

In addition to the network architecture, other hyperparameters are also set in this configuration file. For instance, the number of epochs is set to 50, whereas the optimizer for the training is chosen as ADAM, the batch size is 64, and the learning rate is 0.0005. It is mentioned before that the latent dimension should be smaller than the input dimension so that the anomalies are not reconstructed during the testing phase because of loss of information. Therefore, the latent dimension is also set to 20, thus $(20,1)$ for *discriminatorZ*'s and generator's input shape. Initially similar to encoders, generators,

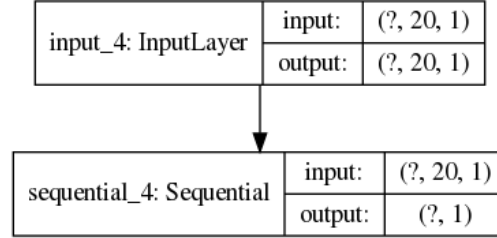


Figure 4.11: The Architecture of the Discriminator Z

and *discriminatorX*'s input shape is also set to (100,51) because of the window size and 51 dimensions of the dataset shown in figure 4.11.

As mentioned before, the variance in preprocessing steps changes the input size, so the neural network should be configured accordingly. Moreover, to find the optimum hyperparameters, grid search is used. Grid search is basically an exhaustive search of a manually chosen subset of a learning algorithm's input space. Grid search algorithms must be directed by a performance metric, which is commonly determined by either cross-validation on the training set or assessment on a validation set. During grid search, the abovementioned parameters and the neural network architecture are changed to find a neural network that detects the most anomalies. The GAN is trained with a subset of the parameters, and the accuracy of the network is measured using the test dataset. The result is saved in a file and then the next subset of hyperparameters is tested. To let these tests run smoothly, the architecture should be able to adapt to any combination of hyperparameters. For instance, the shape of the input to the GAN depends on the window size and the dimensions of the dataset. The dimensions of the dataset vary because every feature extraction or selection algorithm results in a different shape. Since the window size is also a hyperparameter, this variable is set dynamically. The latent dimension is one of the most important hyperparameters. Changing the latent dimension also requires adjustments in encoder, discriminator, and generator networks.

4.5 GAN Training and Reconstruction of the Test Data

The instructor of the GAN class takes the hyperparameter configuration as an argument. It registers these hyperparameters into its attributes. Once the *fit* function is called using the training dataset, the neural network is built using the *_build_tadgan* function. This function calls the *_build_model* function which loops through the layers of networks defined in the configuration file and generates *keras.models.Model* objects for all; encoder, generator, *discriminatorX* and *discriminatorZ* models.

The encoder-generator model can be observed in figure 4.12. During training, the input layer feeds the generator with latent variable z of shape (?,20,1) shown with arrow 1. The generator then reconstructs the $\hat{x}$ of shape (?,100,51) depicted with the arrow 2. In the same way, the encoder's input layer provides the training data of shape (?,100,51)

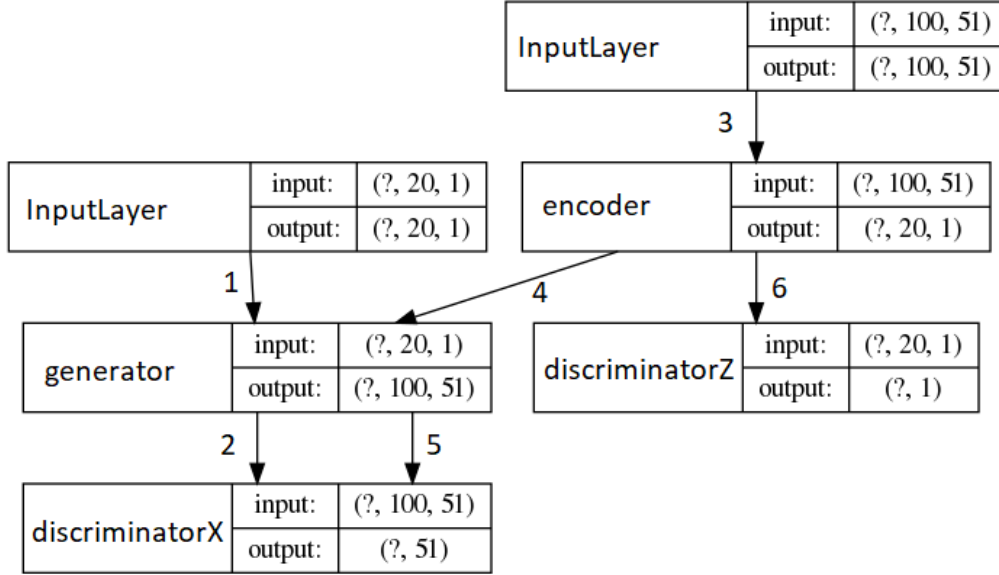


Figure 4.12: The Architecture of the Encoder and Generator

to the encoder depicted with the arrow 3 where the encoder maps the data into latent variables $\hat{z}$ of shape $(?, 20, 1)$. Latent variable $\hat{z}$ is then fed to the *discriminatorZ* and to the generator for reconstruction indicated with the arrows 4 and 6, respectively. In other words, the encoder-generator model receives x and z as inputs and yields reconstructed values $\hat{x}$ as well as outputs of *discriminatorX* and *discriminatorZ*.

The *discriminatorX* model is shown in figure 4.13. The generator's input layer feeds the generator with latent variable z of shape $(?, 20, 1)$ depicted with arrow 1, where the generator reconstructs $\hat{x}$ of shape $(?, 100, 51)$. $\hat{x}$ is then given to the *RandomWeightedAverage* layer as an input along side the training data x . The objective of this is to implement a layer that can do a weighted sum/merge of the outputs of two separate layers, namely the generator and the input layer indicated with the arrows 2 and 3. The output is then fed to the *discriminatorX*, the reconstructed $\hat{x}$ and training data x . To put it another way, the whole model receives x and z as inputs and yields outputs for x , $\hat{x}$ and the product of randomly averaged x and $\hat{x}$.

The architecture of the model *discriminatorZ* is shown in figure 4.14. The encoder is given the training set x of shape $(?, 100, 51)$ as the input by the input layer shown with the arrow 1. The encoder maps the training data to the latent variable $\hat{z}$ of shape $(?, 20, 1)$. Then both $\hat{z}$ and z are given to the *RandomWeightedAverage* layer as input for merging depicted with the arrows 2 and 3 respectively. The output is then given to the final layer along side z and $\hat{z}$. The final layer, *discriminatorZ* yields 3 different outputs corresponding to the inputs. These outputs from the discriminators will form the overall discriminator score.

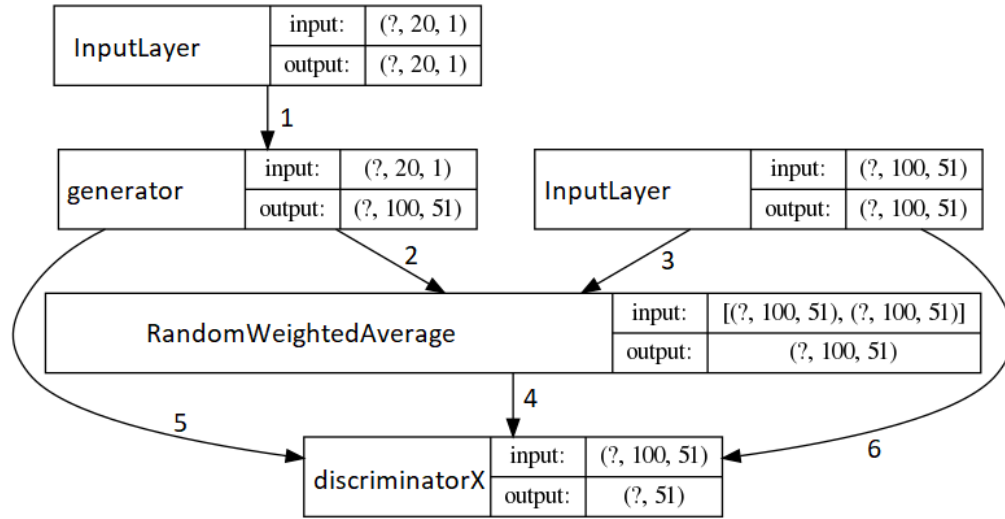


Figure 4.13: The Architecture of the Discriminator for the Reconstructed Data X

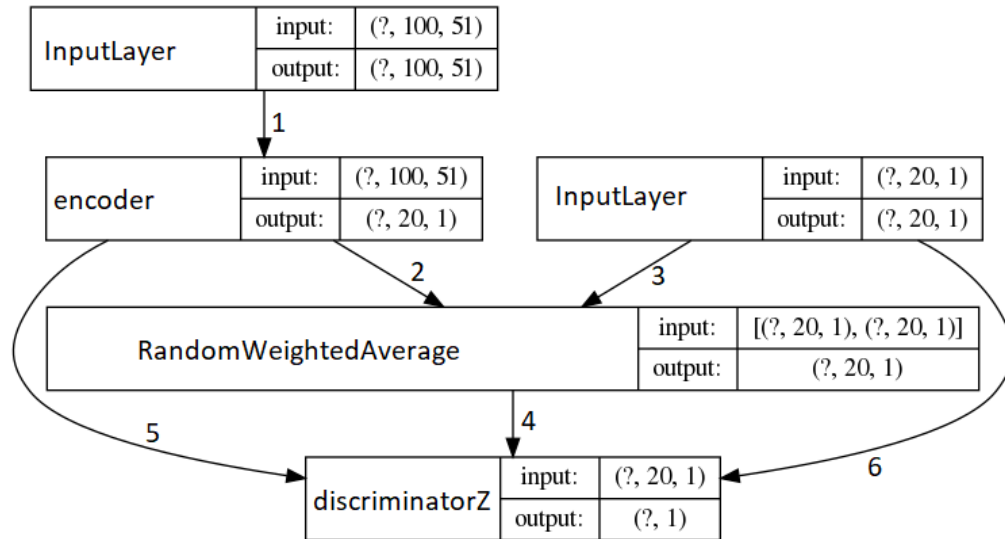


Figure 4.14: The Architecture of the Discriminator for the Reconstructed Data Z

4.5.1 GAN Training

So far, the *fit* method has been triggered but the training has not yet started. Now that the neural network is ready upon the completion of the *_build_tadgan* method, the *_fit* method is triggered to kick start the training. 3 nested for loops are implemented. The outer one loops through epochs, the middle one loops through minibatches, and the inner one loops through discriminator iterations. This is also a hyperparameter indicating how many times discriminators should be trained per minibatch. Generator and discriminator losses are registered into variables each epoch. The training is executed using the method *train_on_batch* from the Keras library, which runs a single gradient update on a single batch of data. In each iteration of minibatch, the losses are returned from the models and appended into the respective arrays. In each iteration of the epoch, the mean of the losses is calculated. It was mentioned earlier that discriminators and encoder-generator models return 3 outputs. After, the mean of these outputs is found for each model, namely *discriminatorX*, *discriminatorZ*, and encoder-generator, these values are printed out and registered into class attributes. Next, the model weights are saved for further use in the future, and *predict* method is ready to be executed.

4.5.2 Reconstruction of the Test Data

The *predict* method simply calls the encoder with the built-in predict functionality, giving the testing data X as input. The returned latent variable $\hat{Z}$ is then fed to the predict function of the generator to reconstruct the dataset $\hat{X}$. Finally, X is given to the predict function of *discriminatorX* to predict the discriminator loss, which will be useful in calculating the anomaly score. The *predict* function returns $\hat{X}$ and discriminator score about this reconstruction.

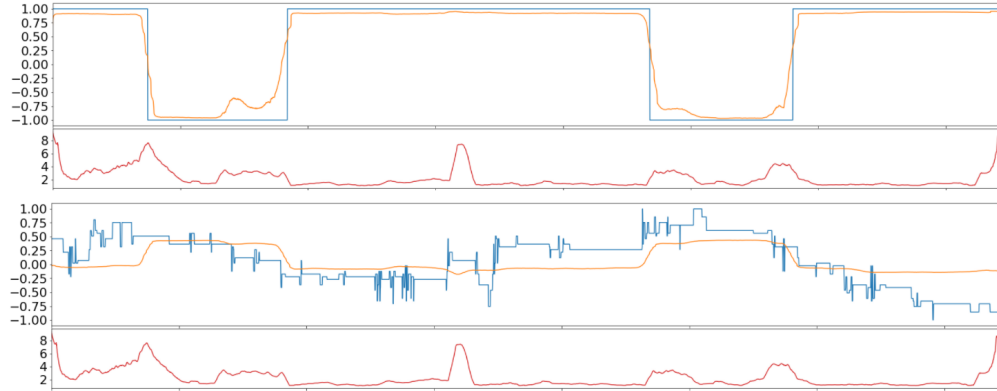


Figure 4.15: Graph of the 2 Dimensions' Prediction and Actual Values of a Time Window

4.6 Anomaly Score Computation

Next, the anomaly score needs to be computed using the function `score_anomalies` where the testing data X , the reconstructed data $\hat{X}$ and discriminator output are given as inputs. First, the reconstruction error is calculated using X and $\hat{X}$. Then this error is multiplied with the discrimination error, which yields the final scores. At this point, predictions for all dimensions are known, which can be compared to the actual test dataset depicted in figure 4.15. The graph with the red line indicates the cumulative difference between the actual and predicted values among all dimensions in a time window. By setting a threshold, time windows can be identified as anomalous if the difference between the reconstructed and the actual data is above the threshold. To set the threshold in a more robust way, `find_anomalies` method is called from the Orion³ library. The resultant time intervals signal that in between these designated time intervals, cyber attacks are being performed. The detection rate can be found by dividing the total count of timestamps that are detected as anomalous by the actual count of anomalous timestamps shown in figure 4.16.

³<https://github.com/sintel-dev/Orion>

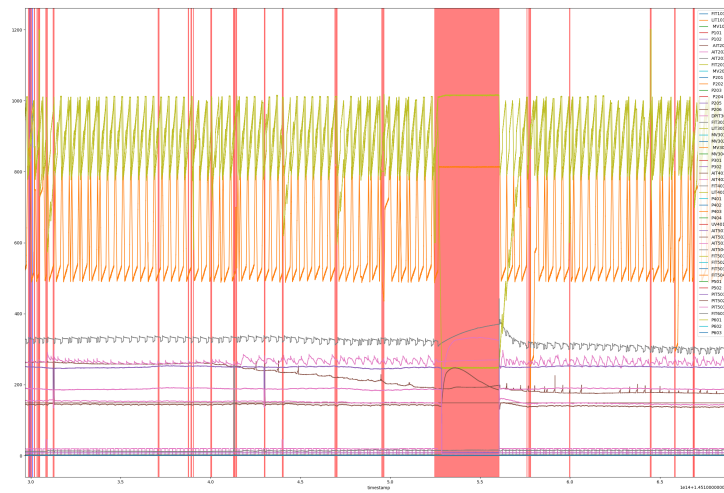


Figure 4.16: SWaT Dataset Indicating Actual and Detected Anomalies

5 Evaluation and Results

Cyber-physical systems are networked physical systems that are often designed to perform mission-critical functions. Water treatment and distribution facilities, natural gas distribution systems, oil refineries, power plants, power grids, and autonomous cars are all examples of CPSs. The development of CPSs for a wide range of activities will be accelerated by the rise of the IOT, resulting in many systems and objects interacting and working independently through networks. As such, cyber-attacks are a major source of worry for CPSs.

Historically, SPC techniques such as CUSUM, EWMA, and Shewhart charts have been used for detecting anomalies. These established detection approaches are unable to deal with the more dynamic and complicated characteristics of CPSs as a result of the Internet of Things. As a result, researchers have gone beyond specification- or signature-based strategies and begun using both supervised and unsupervised machine learning techniques to produce more intelligent and adaptable algorithms for detecting abnormalities or intrusions in large amounts of data.

Even with the application of deep learning algorithms, however, finding abnormalities in time series remains difficult. To begin with, the majority of supervised approaches need sufficient relevant normal data and labeled anomaly classes to learn from, which is rarely the case in reality because of the scarcity of anomalies. Second, although the majority of current unsupervised algorithms are created using linear projection and transformation, the hidden underlying correlations in multivariate time series of complex ICSs are often non-linear. Additionally, the majority of existing algorithms make use of basic comparisons between the current state and anticipated normal ranges, which may be insufficient for anomaly detection since the control boundaries are not sufficiently flexible to discover indirect cyberattacks. For the reasons listed above, there is a need to find robust deep neural networks that not only detect zero-day attacks but can also be trained using multivariate unlabeled data in an unsupervised fashion [6].

There are several promising deep neural network architectures that satisfy such strict requirements. This thesis investigates GAN-based anomaly detectors. The detector contains an encoder, a generator, and two distinguished discriminators that evaluate the performance of the encoder and generator. LSTM cells are used in the layers of deep neural networks to be able to handle time series data. Even though there are an infinite number of combinations to construct such a neural network, this paper takes the neural networks published in MADGAN and TadGAN. On one hand, MADGAN is a multivariate anomaly detection algorithm that has been tested and gave promising results with the SWaT dataset. On the other hand, TadGAN is a univariate anomaly detection algorithm with a sophisticated loss function. This paper investigated the

5 Evaluation and Results

combination of these two state-of-the-art techniques to improve and aimed to contribute to the research. The resulting anomaly detector is tested with the SWaT dataset. SWaT

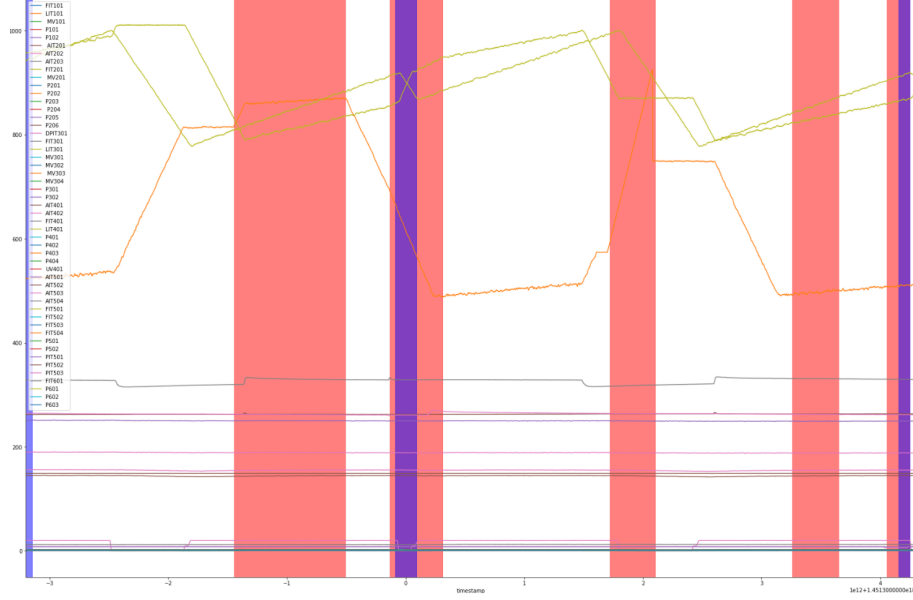


Figure 5.1: SWaT Dataset Indicating Actual and Detected Anomalies

is a fully functional, scaled-down water treatment plant with a compact footprint that produces 5 gallons of double-filtered water per minute. This testbed is modeled after the huge industrial water treatment facilities found in cities. Its primary objective is to allow empirically verified research on the secure and safe design of CPS. SWaT consists of six primary processes that correlate with the facility’s physical and control components. Therefore, it is a widely used testbed in the domain of anomaly detection in industrial control systems, where 35 cyberattacks are implemented. These attacks also vary in types, such as single stage single point, single stage multiple point, multiple stage single point, and multiple stage multiple point.

The GAN is trained using the SWaT dataset, where all three models converged at the end of 50 epochs, as depicted in figure 5.2 5.3 5.4. In other words, the GAN is trained with the data that is collected during the normal functioning of the SCADA system. On the same testbed, 35 different attacks are applied to collect the data. This dataset is used in the testing phase. The GAN is expected to find anomalies in the testing data due to its training with normal data. The SWaT dataset contains 51 dimensions with non-linear relationships between dimensions. Therefore, it is a challenging task to learn the normal behavior pattern of the system and differentiate it from the abnormal behavior. To test if the GAN can correctly identify the timestamps that the cyber attacks are implemented, both known and detected anomalies are drawn on top of each other. The overlapping areas signal that the GAN was able to successfully detect the anomalies. If the red area

does not overlap with the blue area, it signals that the known anomaly is not detected. If the blue area does not overlap with the red area, this signals that an anomaly has been detected, but actually it is normal behavior. This can be observed in figure 5.1 In

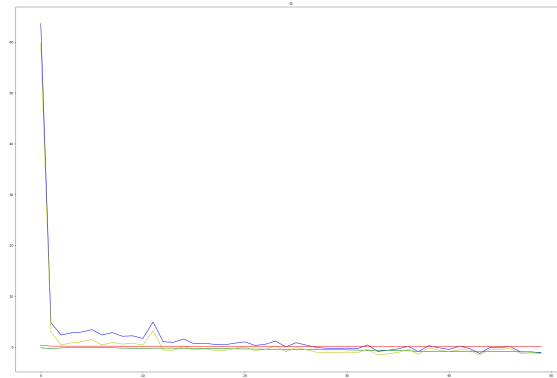


Figure 5.2: Loss Function of the Encoder-Generator Model over 50 Epochs

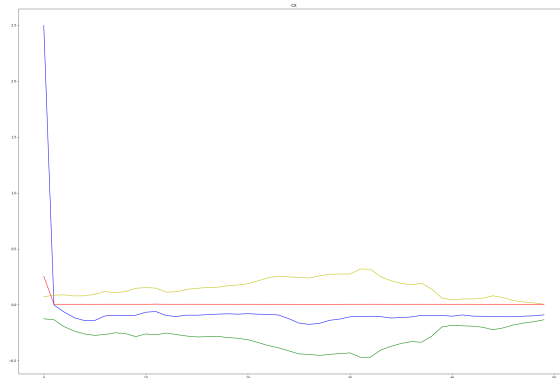


Figure 5.3: Loss Function of the *discriminatorX* Model over 50 Epochs

addition to the verification of the GAN model, its combinations with feature selection and feature extraction methods are investigated. Similar to the findings in MADGAN, the performance is almost the same between the PCA outputting 5 dimensions and the PCA outputting 24 dimensions. The best performance is gathered from the execution where no preprocessing steps are applied, such as feature extraction or feature selection. This can be explained by the characteristics of the model, which is able to learn as

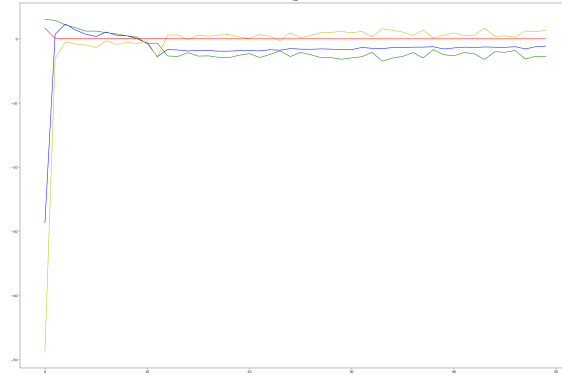


Figure 5.4: Loss Function of *discriminatorZ* Model over 50 Epochs

complex pattern as provided, but the information loss at any rate affects the model in a negative way.

The autoencoder performed the poorest. The number of features is halved using this process. The autoencoder model was shallow compared to the encoder in MADGAN or TadGAN. This might be the cause of the bad performance. The complex non-linear relations between features in the SWaT dataset might not be learned accurately enough to map and express them with half the number of features.

Similar to the autoencoder, feature selection techniques did not also result in adequate results. Unlike autoencoder, it might not be possible to improve these results. In feature selection, some features are selected and the other ones are eliminated. The GAN is then trained with the resulting features. However, there might be some attacks that target these specific eliminated features. As mentioned, there might also be multi-point attacks which target multiple features or sensors. Such cases make it difficult for the GAN to detect anomalies targeted at the eliminated features. Therefore, the autoencoder can be improved with a better architecture, but it is more difficult to improve the feature selection performance.

Among the 5 different feature selection techniques, only LASSO and random forest were able to eliminate features. RFE, chi2 and Pearson correlation were not able to eliminate a single feature and found all features affected the Normal/Attack column. Therefore, the feature selection techniques are not used as a remediation for the curse of dimensionality. One benefit of reducing the dimensions of the dataset is using feature selection is explainability. With fewer features, it is easier to explain or interpret the decisions of the neural network. However, there are also explainable AI techniques like LIME or RISE to reason the decisions of the neural networks that are trained with high-dimensional data.

To sum up, the GAN performed the best without any preprocessing, detecting 475 anomalous timestamps out of 51453. When PCA, which preserves 99% of the variation, is applied as a preprocessing step, 241 out of 51453 timestamps are correctly detected. Finally, when the autoencoder is applied as a feature extraction algorithm before the training, 55 anomalous timestamps are detected out of 51453. Unfortunately, the feature selection algorithm detected 0 anomalous timestamps, so the results were not worth investigating further.

It is observed that the bigger the input window, the more accurate the model is. This is unsurprising, since more inputs supply the model with more unique input signatures, simplifying the job of mapping inputs to training data outputs. Due to the high level of noise in the data, a single input window can hardly learn to identify a single abnormal timestamp. Unsurprisingly, increasing the window size from 1 to 50 enables the model to learn a more accurate representation of the recovered data. However, raising the window size to 100 or 200 results in a performance degradation. A window with a size of 200 is incapable of detecting anything. Of the ten window sizes examined for this specific combination of data and neural network model, an input window size of 60 provides the best performance on the SWaT data. Since the SWaT data is recorded every second, the window length of 60 is not surprising indicating a minute. On one hand, it is shown that expanding the input window improves model performance. On the other hand, a window that is excessively big may result in over-fitting, thus reducing model performance. The appropriate window size depends on the data and model used in each situation. Since the model and data change dynamically during the execution of the algorithm depending on the outcome of the preprocessing steps, it is challenging to find a generalizable optimal solution for the window size. However, according to the tests, the best step size is found to be 1 when using the rolling window function.

To conduct these tests, the euclid server is provided by TUM. It has an AMD EPYC 7551P 32-Core processor, 125 GB of RAM, and a TITAN X 12GB GPU to improve the performance of the neural network during training and testing.

5.1 MADGAN and GAN-AD for Benchmark

To compare the performance, MADGAN and GAN-AD are implemented as benchmarks. Both techniques are trained and tested using the same hardware and dataset. MADGAN applies PCA, which results in a dataset with 5 dimensions. Therefore, the results of MADGAN are compared with the outcome of the preprocessing and training using PCA. MADGAN has a 61% recall and GAN-AD has a 53% recall, whereas our GAN has only 4%. Even though models perform much better than our GAN, they also generate much higher false positive rates, 37% and 58% for MADGAN and GAN-AD, respectively. In contrast, our GAN performs at only 1%. The reason why MADGAN and GAN-AD do not perform as well as claimed in the paper could be because of hyperparameter fine tuning. The configuration file that is given in their github repository is used when training both MADGAN and GAN-AD and no further hyperparameter optimization

steps are applied. Nevertheless, the low false positive rate gathered from our GAN raises some questions for future research and makes this research promising.

6 Discussion

It is mentioned that PCA is used as a feature extraction technique to reduce dimensionality while keeping the information in terms of variance in the dataset. Even though this technique yields the best results among the feature extraction and selection techniques, it also has disadvantages. By using PCA, the column names are lost. Since PCA extracts new features in place of the old ones, interpretation of the results becomes more difficult. Beforehand, the sensor and actuator names were the labels of the columns, which were replaced with generated numbers. Explainability and interpretability can be crucial if the system at hand executes critical tasks that can affect human life. In these situations, engineers might want to know the reasoning behind the decision made by a neural network when losing initial feature names can make the situation more challenging. Moreover, the GAN cannot identify the roots of the anomalies in the CPS, because the name of the attacked sensor is replaced by the generated features by PCA.

One reason why the resulting GAN performed better than MADGAN and GAN-AD in terms of false negatives can be the PCA dimension. In paper MADGAN, PCA is used as a feature extraction method which yields 5 dimensions for the SWaT dataset. In this thesis, PCA is used to have 24 dimensions. The difference of variance between 5 and 24 dimensions is not less than 3%. However, using more dimensions has an overlooked benefit. Using more principal components could reduce false negatives due to the noise information brought in by adding less important PC dimensions.

Features are eliminated to refine the training data using 5 feature selection techniques. However other than LASSO and random forest, none of the techniques were able to eliminate any feature. Since all these methods are supervised feature selection techniques, test data is used. In figure 4.4, the first table shows that no feature is eliminated because no attacks are implemented in training data. The second table shows the eliminated features using the test data which have less than 3 points in Total column. However, at the end of the pipeline, no anomalies are found when feature selection is used. Therefore feature selection techniques are observed to be ineffective in this task.

As stated in the evaluation chapter, the resulting GAN did not perform adequately. The reason for this could be dynamically setting the neural network architecture. Depending on the outcome of the preprocessing steps, the neural network was programmed to adapt to the input. Using some domain knowledge at this point can improve the performance. For instance, the latent dimension should be less than the window size so that the anomalies are not mapped to the reconstructed data due to information loss. However, depending on the preprocessing step, the initial dimension can result in 5, which would be equal to the latent dimension. This process can be optimized by setting up some rules based on domain knowledge.

6.1 Limitations

Domain knowledge can also help define the best neural network architecture. However, it is aimed to avoid using domain knowledge to find a generalizable solution that can be applied to multiple problems. A more extensive grid search can be executed to find the optimum neural network architecture. The latent dimension subspace could not be investigated further to find the best latent dimensions window size combination to increase the performance. Due to resource limitations such as time and computing power, it was not possible to investigate more into the matter. Since the neural network allocated 98% of a single GPU, multiprocessing was not possible. Because running grid search sequentially did not produce an efficient result, the search space for the subset of hyperparameters is kept limited. This is the main reason why the resulting GAN architecture and pipeline did not perform better than the other benchmarks.

An autoencoder was used as a feature extraction method, which did not result in a good anomaly detection score. The autoencoder architecture can be tailored for such a feature extraction case. No further investigation was conducted since the training and testing accuracy were adequate. However, the resulting performance issue might be because the autoencoder was too shallow for such a complex dataset and was not able to learn all relations between features.

6.2 Future Works

A more extensive grid search can be conducted for both the autoencoder and the resultant GAN for anomaly detection. It is believed that hyperparameter tuning does not significantly alter the performance. However, in the case of GAN for anomaly detection, the hyperparameter values change the results dramatically. Even a change of 10 in window size or latent dimension can result in no anomaly detection from the whole test set. Therefore, it is worthwhile to investigate the hyperparameter space more, which looks promising.

In addition to the autoencoder or PCA, there are other feature extraction methods as well. Independent Component Analysis (ICA), t-distributed Stochastic Neighbor Embedding (t-SNE), locally linear Embedding (LLE), or denoising autoencoder, variational autoencoder, convolutional autoencoder, and sparse autoencoder are a few of them [39].

ICA is a linear dimensionality reduction technique that accepts a combination of independent components as input data and attempts to accurately identify each one (deleting all the unnecessary noise). Two input characteristics are said to be independent if their linear and nonlinear dependencies are equal to zero. Independent Component Analysis is often used in medical applications such as electroencephalography (EEG) and functional magnetic resonance imaging (fMRI) to distinguish valuable signals from unhelpful ones.

Locally Linear Embedding is a method for reducing the dimension of a dataset based on Manifold Learning. A manifold is a two-dimensional item that is embedded in a

three-dimensional space. Manifold Learning then represents this element in its original D dimensions rather than a larger area that is unnecessary.

The t-SNE algorithm minimizes the divergence between a distribution defined by the pairwise probability similarities of the input features in the original high-dimensional space and its reduced low-dimensional counterpart. The Kullback-Leiber (KL) divergence is then used to quantify the dissimilarity between two distinct distributions in t-SNE. Gradient descent is then used to reduce the KL divergence. When t-SNE is used, a Gaussian distribution is used to represent the higher-dimensional space, while a Student's t-distribution is used to describe the lower-dimensional space. This is done to prevent an imbalance in the distance distribution between adjacent locations generated by the translation into a lower-dimensional space.

In addition to these techniques, it is already mentioned that it is worth investigating more and different kinds of autoencoders and their hyperparameter spaces. Using multiple GPUs in a parallel fashion can accelerate the search of hyperparameter subspaces to find optimal parameters for the neural networks.

7 Conclusion

This thesis aims to develop a GAN-based anomaly detector that can be employed in industrial control systems. One of the characteristics of industrial control systems is that many actuators and sensors hold complex relationships with each other, which creates challenges during learning. Due to these many features, the GAN should be able to learn from multivariate data. Another feature of such datasets is that they are frequently unbalanced and unlabeled. Therefore, the training of the GAN is required to be unsupervised. Since there are more normal timestamps than anomalous timestamps, the GAN should also be prone to imbalanced datasets.

Another contribution aimed at by this thesis is the investigation of the window size and the preprocessing methods that can be applied before the neural network training to remediate the curse of dimensionality. For feature selection, techniques like Pearson correlation, chi2 test, RFE, LASSO, and random forest are used. The features that were not eliminated by one of these methods collected a point, and the features that collected the least points were taken out of the dataset for the training. Moreover, PCA and autoencoder techniques are studied and tested as feature extraction techniques. Tests using PCA resulted in the best anomaly score, followed by no preprocessing and autoencoder.

MADGAN and TadGAN papers are used as the starting point in this thesis to construct a GAN-based anomaly detector. The loss functions, presented in TadGAN, are used by leveraging the MADGAN's multivariate characteristics to improve the GAN architecture. Since the input shape varied depending on the preprocessing steps taken, it was challenging to find the optimum window size. In addition, the hyperparameters greatly influenced the window size, such as the size of the latent dimension, the step size while creating windows, or the batch size.

This thesis used the papers, MADGAN and GAN-AD as the benchmark. Interestingly, the performance of these architectures did not meet the expectations as stated in their respective papers. This signals to us the importance of hyperparameter tuning and the effects of small tweaks on the neural network architectures. When implementing the above-stated neural networks, the default configuration is used from their github pages. Therefore, this can be one of the reasons why they performed poorly, and this can also be the reason why the GAN subject to this thesis did not meet the expectations. However, it is worthwhile to investigate hyperparameter space, additional feature extraction techniques and their effects on the neural network, as well as a set of rules for dynamically adjusting the neural network during execution based on the input dataset.

List of Figures

2.1	ICS Architecture	7
2.2	LSTM Cell Architecture	15
2.3	TadGAN Architecture	19
4.1	GAN Pipeline	38
4.2	SWaT Normal Dataset Comparison	39
4.3	SWaT Attack Dataset Comparison	39
4.4	Feature Selection Table	41
4.5	Autoencoder Architecture	46
4.6	Autoencoder Loss Function	48
4.7	Rolling Window Schema	49
4.8	The Architecture of the Encoder	50
4.9	The Architecture of the Generator	51
4.10	The Architecture of the Discriminator X	51
4.11	The Architecture of the Discriminator Z	52
4.12	The Architecture of the Encoder and Generator	53
4.13	The Architecture of the Discriminator for the Reconstructed Data X	54
4.14	The Architecture of the Discriminator for the Reconstructed Data Z	54
4.15	Graph of the 2 Dimensions' Prediction and Actual Value of a Time Window	56
4.16	SWaT Dataset Indicating Actual and Detected Anomalies	57
5.1	SWaT Dataset Indicating Actual and Detected Anomalies	60
5.2	Loss Function of the Encoder-Generator Model over 50 Epochs	61
5.3	Loss Function of the <i>discriminatorX</i> Model over 50 Epochs	61
5.4	Loss Function of <i>discriminatorZ</i> Model over 50 Epochs	62

Bibliography

- [1] P. Schneider and K. Böttinger. “High-Performance Unsupervised Anomaly Detection for Cyber-Physical System Networks”. In: Oct. 2018, pp. 1–12. DOI: 10.1145/3264888.3264890.
- [2] D. Li, D. Chen, L. Shi, B. Jin, J. Goh, and S.-K. Ng. *MAD-GAN: Multivariate Anomaly Detection for Time Series Data with Generative Adversarial Networks*. 2019. arXiv: 1901.04997 [cs.LG].
- [3] H. Zenati, M. Romain, C. S. Foo, B. Lecouat, and V. R. Chandrasekhar. “Adversarially Learned Anomaly Detection”. In: *CoRR* abs/1812.02288 (2018). arXiv: 1812.02288. URL: <http://arxiv.org/abs/1812.02288>.
- [4] W. Jiang, Y. Hong, B. Zhou, X. He, and C. Cheng. “A GAN-Based Anomaly Detection Approach for Imbalanced Industrial Time Series”. In: *IEEE Access* 7 (2019), pp. 143608–143619.
- [5] A. Geiger, D. Liu, S. Alnegheimish, A. Cuesta-Infante, and K. Veeramachaneni. “TadGAN: Time Series Anomaly Detection Using Generative Adversarial Networks”. In: *CoRR* abs/2009.07769 (2020). arXiv: 2009.07769. URL: <https://arxiv.org/abs/2009.07769>.
- [6] D. Li, D. Chen, J. Goh, and S. Ng. “Anomaly Detection with Generative Adversarial Networks for Multivariate Time Series”. In: *CoRR* abs/1809.04758 (2018). arXiv: 1809.04758. URL: <http://arxiv.org/abs/1809.04758>.
- [7] H. Zenati, C. S. Foo, B. Lecouat, G. Manek, and V. R. Chandrasekhar. “Efficient GAN-Based Anomaly Detection”. In: *CoRR* abs/1802.06222 (2018). arXiv: 1802.06222. URL: <http://arxiv.org/abs/1802.06222>.
- [8] S. Adepu, V. R. Palleti, G. Mishra, and A. Mathur. “Investigation of Cyber Attacks on a Water Distribution System”. In: *CoRR* abs/1906.02279 (2019). arXiv: 1906.02279. URL: <http://arxiv.org/abs/1906.02279>.
- [9] J. Suaboot, A. Fahad, Z. Tari, J. Grundy, A. Mahmood, A. Almalawi, A. Zomaya, and K. Drira. “A Taxonomy of Supervised Learning for IDSs in SCADA Environments”. In: *ACM Computing Surveys* 53 (Apr. 2020), p. 37. DOI: 10.1145/3379499.
- [10] Rahil Shaikh. *Feature Selection Techniques in Machine Learning with Python*. <https://towardsdatascience.com/feature-selection-techniques-in-machine-learning-with-python-f24e7da3f36e>. Accessed: 2021-09-20. 2018.
- [11] I. Guyon and A. Elisseeff. “An Introduction to Variable and Feature Selection”. In: *J. Mach. Learn. Res.* 3.null (2003), pp. 1157–1182. ISSN: 1532-4435.

- [12] A. Holmes, B. Illowsky, S. Dean, and O. College. *Introductory Business Statistics*. OpenStax, 2017. ISBN: 9781947172463. URL: https://books.google.de/books?id=vr%5C_GtAEACAAJ.
- [13] ADAM HAYES. *Chi-Square (2) Statistic*. <https://www.investopedia.com/terms/c/chi-square-statistic.asp>. Accessed: 2021-09-20. 2021.
- [14] Jason Brownlee. *Recursive Feature Elimination (RFE) for Feature Selection in Python*. <https://machinelearningmastery.com/rfe-feature-selection-in-python/>. Accessed: 2021-09-20. 2020.
- [15] B. Butcher and B. J. Smith. "Feature Engineering and Selection: A Practical Approach for Predictive Models". In: *The American Statistician* 74.3 (2020), pp. 308–309. doi: 10.1080/00031305.2020.1790217. eprint: <https://doi.org/10.1080/00031305.2020.1790217>. URL: <https://doi.org/10.1080/00031305.2020.1790217>.
- [16] Akash Dubey. *Feature Selection Using Random forest*. <https://towardsdatascience.com/feature-selection-using-random-forest-26d7b747597f>. Accessed: 2021-09-20. 2018.
- [17] Gianluca Malato. *Feature selection in machine learning using Lasso regression*. <https://towardsdatascience.com/feature-selection-in-machine-learning-using-lasso-regression-7809c7c2771a>. Accessed: 2021-09-20. 2021.
- [18] A. L. Gomez, L. Fernández-Maimó, A. Huertas, and F. J. García Clemente. "MADICS: A Methodology for Anomaly Detection in Industrial Control Systems". In: *Symmetry* 12 (Sept. 2020), p. 1583. doi: 10.3390/sym12101583.
- [19] J. Forkman, J. Josse, and H.-P. Piepho. "Hypothesis Tests for Principal Component Analysis When Variables are Standardized". In: *Journal of Agricultural, Biological and Environmental Statistics* 24 (Feb. 2019). doi: 10.1007/s13253-019-00355-5.
- [20] Q. Meng, D. R. Catchpoole, D. B. Skillicorn, and P. J. Kennedy. "Relational Autoencoder for Feature Extraction". In: *CoRR* abs/1802.03145 (2018). arXiv: 1802.03145. URL: <http://arxiv.org/abs/1802.03145>.
- [21] rajkumar. *Time Series Predictions with LSTM*. <https://medium.datadriveninvestor.com/time-series-predictions-with-lstm-4877c38d87d1>. Accessed: 2021-09-20. 2019.
- [22] M. Leznik, P. Michalsky, P. Willis, B. Schanzel, P.-O. Östberg, and J. Domaschka. "Multivariate Time Series Synthesis Using Generative Adversarial Networks". In: Apr. 2021, pp. 43–50. doi: 10.1145/3427921.3450257.
- [23] Z. Lin, A. Jain, C. Wang, G. Fanti, and V. Sekar. *Generating High-fidelity, Synthetic Time Series Datasets with DoppelGANger*. Sept. 2019.
- [24] J. Goh, S. Adepu, K. Junejo, and A. Mathur. "A Dataset to Support Research in the Design of Secure Water Treatment Systems". In: Oct. 2016.

-
- [25] J. Goh, S. Adepu, Y. X. M. Tan, and Z. Lee. "Anomaly Detection in Cyber Physical Systems Using Recurrent Neural Networks". In: Jan. 2017, pp. 140–145. doi: 10.1109/HASE.2017.36.
- [26] D. Antonioli, A.-R. Sadeghi, N. O. Tippenhauer, and H. Ghaeini. *State-Aware Anomaly Detection for Industrial Control Systems*. Apr. 2018. doi: 10.1145/3167132.3167305.
- [27] C. P. Ngo, A. A. Winarto, C. K. L. Kou, S. Park, F. Akram, and H. K. Lee. "Fence GAN: Towards Better Anomaly Detection". In: *CoRR* abs/1904.01209 (2019). arXiv: 1904.01209. URL: <http://arxiv.org/abs/1904.01209>.
- [28] J. Inoue, Y. Yamagata, Y. Chen, C. Poskitt, and J. Sun. "Anomaly Detection for a Water Treatment System Using Unsupervised Machine Learning". In: Nov. 2017, pp. 1058–1065. doi: 10.1109/ICDMW.2017.149.
- [29] M. Du, F. Li, G. Zheng, and V. Srikumar. "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning". In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS '17*. Dallas, Texas, USA: Association for Computing Machinery, 2017, pp. 1285–1298. ISBN: 9781450349468. doi: 10.1145/3133956.3134015. URL: <https://doi.org/10.1145/3133956.3134015>.
- [30] M. Kravchik and A. Shabtai. "Detecting Cyber Attacks in Industrial Control Systems Using Convolutional Neural Networks". In: Oct. 2018, pp. 72–83. doi: 10.1145/3264888.3264896.
- [31] C. Feng, T. Li, and D. Chana. "Multi-level Anomaly Detection in Industrial Control Systems via Package Signatures and LSTM Networks". In: *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 2017, pp. 261–272. doi: 10.1109/DSN.2017.34.
- [32] M. Landauer, M. Wurzenberger, F. Skopik, G. Settanni, and P. Filzmoser. "Time Series Analysis: Unsupervised Anomaly Detection Beyond Outlier Detection: 14th International Conference, ISPEC 2018, Tokyo, Japan, September 25-27, 2018, Proceedings". In: Jan. 2018, pp. 19–36. ISBN: 978-3-319-99806-0. doi: 10.1007/978-3-319-99807-7_2.
- [33] W. Ren, T. Yardley, and K. Nahrstedt. "EDMAND: Edge-Based Multi-Level Anomaly Detection for SCADA Networks". In: Oct. 2018, pp. 1–7. doi: 10.1109/SmartGridComm.2018.8587533.
- [34] C. U. Carmona, F. Aubet, V. Flunkert, and J. Gasthaus. "Neural Contextual Anomaly Detection for Time Series". In: *CoRR* abs/2107.07702 (2021). arXiv: 2107.07702. URL: <https://arxiv.org/abs/2107.07702>.
- [35] G. Zizzo, C. Hankin, S. Maffei, and K. Jones. "Adversarial Attacks on Time-Series Intrusion Detection for Industrial Control Systems". In: *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. 2020, pp. 899–910. doi: 10.1109/TrustCom50675.2020.00121.
-

- [36] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* (2011).
- [37] Markus Sun. *Time Series Feature Extraction for Industrial Big Data (IIoT) Applications*. <https://techairesearch.com/time-series-feature-extraction-for-industrial-big-data-iiot-applications/>. Accessed: 2021-11-20. 2019.
- [38] *tensorflow tf.keras.layers.BatchNormalization*. https://www.tensorflow.org/api_docs/python/tf/keras/layers/BatchNormalization. Accessed: 2021-11-20. 2021.
- [39] Pier Paolo Ippolito. *Feature Extraction Techniques*. <https://towardsdatascience.com/feature-extraction-techniques-d619b56e31be>. Accessed: 2021-11-20. 2019.